

ROS2 Introduction

Table of Contents

1. Introduction

1.1. Challenges with autonomous robots

1.2. ROS2, a middleware between software and hardware

1.3. ROS2 main concept

1.4. ROS2: Packages for autonomous robots

2. Appendix

1. Introduction

ROS2 is middleware that can run on a PC or an on-board computer (e.g., Raspberry Pi or Jetson). It comes with a steep learning curve, but it is useful for building intelligent robots, unlike teleoperated robots (e.g., Arduino robots).

- **ROS2** stands for Robotic Operating System, and it is the second version. The term *operating system* is confusing because ROS2 does not include a graphical interface like Windows or MacOS, and it mainly uses terminal commands. However, one of the first tasks of operating systems is to connect peripherals, e.g., a mouse, keyboard, and screen. In the case of robotics, ROS2 connects motors, sensors, and their dedicated algorithms.
- The first version of ROS is still quite widely used, but it does not scale well with the robot fleet, hence the development of ROS2.
- **ROS2 is a middleware:** it is the part that stands between the hardware and the software. ROS2 is similar to a driver for peripherals: e.g., when you plug a mouse into a computer, it just works instantaneously. The drivers are installed automatically as soon as the device is plugged in, and you do not write any code that converts the optical flow sensor (mouse sensor) to movement in the x-y direction. Your computer understands the information provided by the mouse directly, thanks to the standardization of the communication protocol via USB.

1.1. Challenges with autonomous robots

The following videos were developed by Norlab, in French, but with English subtitles available. The laboratory Norlab focuses on autonomous field robots in a northern, harsh environment.

- History of autonomous vehicles:
<https://youtu.be/-vKPvJXooNs?si=iUMQm3fQmWnBT1B5>

- What is a mobile robot?

https://youtu.be/PJn7bch13Yo?si=kv2qXKYrSWB_zeJb

A robot needs sensors to perceive the environment, algorithms to treat this information, and actuators to act accordingly.

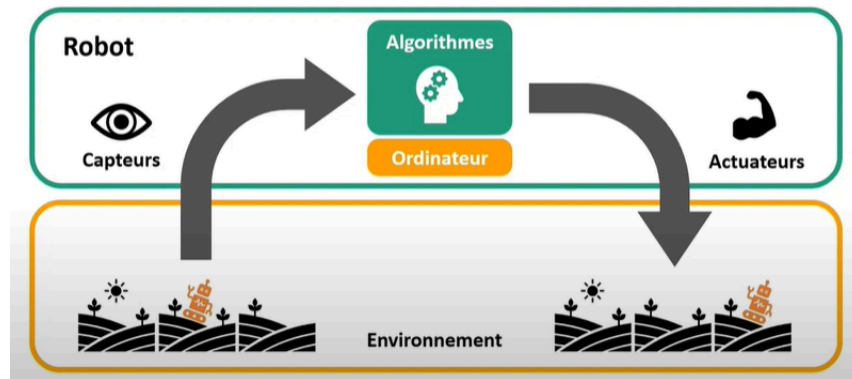


Figure 1: An autonomous robot requires sensors, actuators, and algorithms

A robot has several peripherals (generic names for sensors and actuators, similar to peripherals for a computer, e.g., mouse, keyboard, screen, speakers, microphone, etc.), with different algorithms that are intense and quite consuming to develop, such as:

- Code interfacing between the hardware and the computer (i.e., a driver)
 - o Sensors: Lidar, radar, sonar, camera, GPS, IMU, encoder
 - o Actuators: DC, BLDC, steppers
- Algorithms to interpret the sensors' data:
 - o Computer vision with a camera sensor
 - o Mapping/SLAM with Lidar, radar, and sonar sensors
 - o Positioning/navigation with GPS, IMU, and encoder
 - o Digital twin – Visualization/simulation of the robot on the computer

For simple tasks, such as teleoperation of a wheeled robot, microcontrollers based on Arduino/esp32 are fine. Commands from joysticks are translated to signals fed to the robots' motors. However, complex tasks involve more hardware, communication protocols, and computation. For instance, a typical autonomous robot involves four wheels, four encoders, one depth camera with a computer vision algorithm, lidar with mapping (SLAM), swarm/decentralized communication for multiple robots, and texture mapping with a depth 3D camera: there is no way to implement this into an Arduino robot.

1.2. ROS2, a middleware between software and hardware

In French, with English subtitles available.

- Why ROS2? <https://youtu.be/vAb5SnaJbF0?si=bhGIA70abbXpxJWM>
- Camera and vision: <https://youtu.be/hfCB2IRdQmg?si=FS-GwnXfUbptW-K2>
- Modeling and control: <https://youtu.be/UhVIXqckmv4?si=St-mZjxOBteLqQ2Q>
- Lidar and SLAM: <https://youtu.be/nqcmNkzTbM4?si=djCLtcyxlNUTXqgH>

The development of robotic systems is a worldwide activity. Thus, robotic engineers share common problems, such as drivers for hardware, navigation, SLAM, and computer vision, etc.

- ROS2 is based on the systems of **nodes** (pieces of individual code) that communicate with channels called **topics**.
- It uses two programming languages: Python for prototyping, analysis, and communication between nodes. The other language, C++, is used for real-time (microcontroller).
- The greatest benefit of ROS2 is code sharing: you can decide to write down the computer vision algorithm all over again, or to use a library that has already been developed; it requires work regarding the inputs, outputs, and parameters of the packages, but it is much less prone to errors relative to coding the algorithms by yourself.

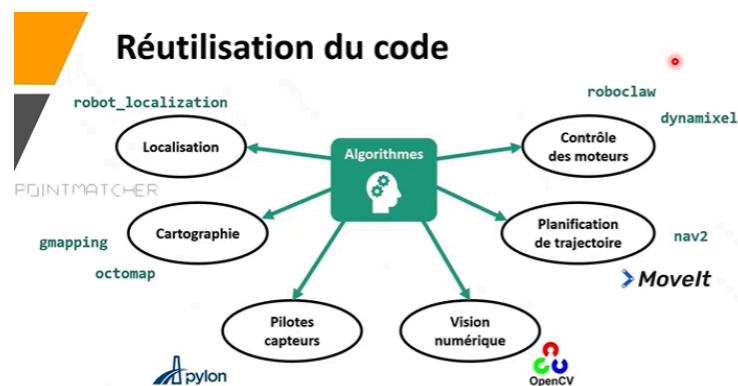


Figure 2: Autonomous vehicles must handle several tasks: mapping, navigation, computer vision, etc. Code sharing becomes even more important as the robot is becoming more autonomous and intelligent.



Figure 3: ROS2 aims at creating a standard for all robotic platforms: they can thus be summarized as black boxes receiving inputs and transmitting outputs.

Outils intégrés

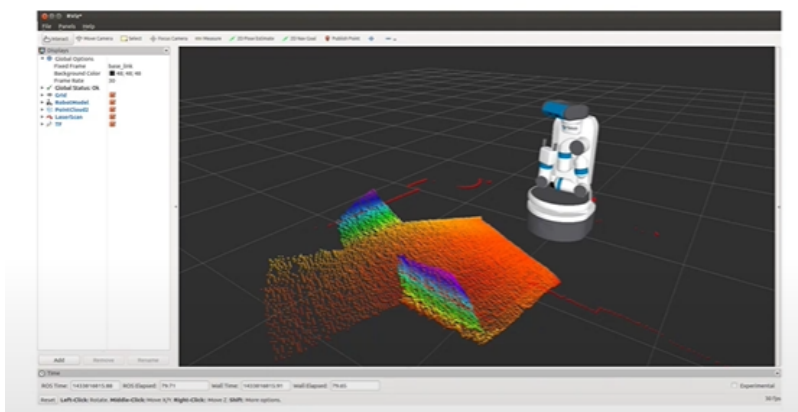


Figure 4: Digital Twin (RViz and Gazebo) - ROS2 offers tools for visualization of data coming from the hardware, which is time-consuming if one needs to develop it entirely.

1.3. ROS2 main concept

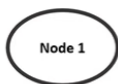
ROS2 aims at creating a standard for different hardware, software, and engineers. Thus, the ROS2 framework should be understood first.

- **Node:** It is a block of code. For example, inverse kinematics takes Cartesian position x, y, z as input and outputs joint angles.
- **Topic:** Data, e.g., encoders, imu, lidar, etc. Nodes subscribe to a topic (it receives information from the topic as input) or publish to a topic (it outputs to the topic).
- ROS2 uses the **Terminal:** one must become comfortable with the command line.

- **URDF** stands for Unified Robot Description Format: it describes the robot's dimensions and types of joints in an XML format. This is an essential part of ROS2, as one must perform simulation and visualization of the actual robot.
- **A package** is a collection of nodes, topics, and libraries that perform a specific task, e.g., a computer vision package, SLAM package, etc. Thus, one must understand the concept of cloning, compiling, and utilizing a package.
- **Launch files** are a Python script to launch the required packages for your robot.

Nodes

- Élément de base de ROS
- Bloc de code qui effectue une tâche spécifique
- Indépendant des autres nodes



Topics

- Canaux de communication entre les nodes
- Échange un type de message précis
- Les nodes "s'abonnent" et "publient" sur des topics



Figure 5: Nodes and Topics are the cornerstone of ROS2.

URDF

- Fichier de description du robot
- Définition des TFs entre les modules
- Visualisation 3D
- Caractérisation physique des modules pour la simulation (ex: inertie, masse, volume)



Figure 6: URDF is essential for describing a robot in ROS2 and especially for RVIZ

Packages

- Regroupement de code ROS réutilisable
- Peut contenir :
 - Nodes
 - Librairies externes
 - Fichiers de configuration
 - URDF
 - Messages personnalisés
 - +++
- Clone -> Compile -> Utilise



Figure 7: ROS2 is about integrating packages

Launch files

- Configuration du système
- Lancer plusieurs nodes en même temps

```
ros2 launch <package_name>
<launch_file_name>
```

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            namespace='turtlesim1',
            executable='turtlesim_node',
            name='sim1'
        ),
        Node(
            package='turtlesim',
            namespace='turtlesim2',
            executable='turtlesim_node',
            name='sim2'
        ),
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='sim3',
            remappings=[
                ('/input/pose', '/turtlesim1/turtle1/pose'),
                ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
            ]
        )
    ])

```

Figure 8: Launch file: a bit similar to a MATLAB script: you launch the nodes

1.4. ROS2: Packages for autonomous robots

The general structure of an autonomous robot is given in the following table, which represents the minimum base for autonomous robots. Additional features and sensors are presented in the advanced concepts.

Name	Description
rmitbot_description	- contains URDF and Gazebo configuration - uses Gazebo and RViz (a package for visualization) - The virtual robot cannot move yet.
rmitbot_controller	- uses ros2_control and ros2_controllers (a package for controlling the robot) - interface the joystick/keyboard controller to the joints of the robot - The virtual robot can move; its motion can be measured with odometry. Adding an IMU would attenuate the error due to slippage.
rmitbot_localization	- uses robot_localization (package for EKF) - performs sensor fusion (EKF) between odometry and imu - The virtual robot can move, and its pose can be displayed
rmitbot_mapping	- Create a map of the environment with slam_toolbox
rmitbot_navigation	- path planning (a_star): path to get to the target - motion planning: path following with PID - navigation: decision tree relative to known and unknown obstacles
rmitbot_vision	- includes camera plugins
rmitbot_firmware	- Includes hardware interface (communication between rpi and esp32)

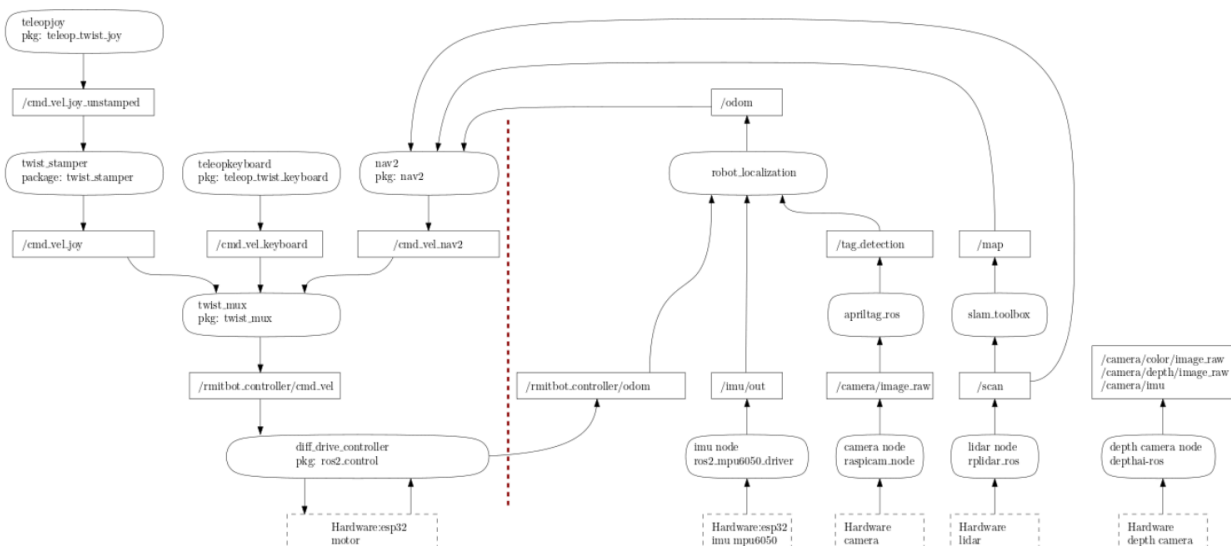
The workspace tree should look like the following:

rmitbot_ws

→ _ws stands for workspace, where you develop your

code.

- build, install, log, .vscode: → These folders are automatically created while compiling the ws
- src → The source folder contains all the packages used
 - rmitbot_bringup
 - rmitbot_description
 - rmitbot_controller
 - rmitbot_localization
 - rmitbot_mapping
 - rmitbot_navigation
 - rmitbot_firmware
 - rmitbot_vision



2. Appendix

Useful information:

- Articulated Robotics: <https://youtu.be/XcJzOYe3E6M?si=HZpo33T DFA1Cuja0>
<https://articulatedrobotics.xyz/tutorials/docker/docker-overview>
- F1 Tenth at UPenn: https://youtu.be/EU-QaO6xTv4?si=LN_v0i6MaEkdJbco
- <https://github.com/AntoBrandi/Bumper-Bot>
- <https://github.com/ros-mobile-robots/diffbot> (this is ROS1 though)

- <https://automaticaddison.com/naming-and-organizing-packages-in-large-ros-2-projects/>
- <https://github.com/aws-deepracer/aws-deepracer/blob/main/introduction-to-the-ros-navigation-stack-using-aws-deepracer-evo.md>

ROS2 Preparation for PC and RPI

Table of Contents

[1. Introduction](#)

[2. Preparing the computer](#)

[2.1. Using Ubuntu via Virtual Machine](#)

[2.2. Using Ubuntu with an SSD Flash Drive](#)

[2.3. Installing ROS2 Jazzy](#)

[2.4. Installing additional software](#)

[3. Preparing the RPI](#)

[3.1. Using Raspberry Pi Imager and installing Ubuntu on the RPI](#)

[3.2. Installing ROS2 Jazzy on the RPI](#)

[4. SSH via terminal: Testing the communication between RPI and PC](#)

[5. SSH via VSCode](#)

[6. Going further](#)

1. Introduction

This section introduces the preparation of the physical setup:

- PC with Ubuntu 24.04 **Desktop** and ROS2 Jazzy **full install**
- RPI5 with Ubuntu 24.04 **server** and ROS2 Jazzy **bare bones**

2. Preparing the computer

ROS2 works much better on Ubuntu. Windows and MacOS are supported, but not as much as with Ubuntu. There are three ways of installing Ubuntu:

- Virtual Machine: Windows uses VirtualBox to run Ubuntu 24.04. Works well, but the simulation lags.
- Dual-boot: This requires modification on your PC, which can be complicated.
- SSD USB Stick boot: the most modular solution. Simulation works well, and no modification of your PC is required.

2.1. Using Ubuntu via Virtual Machine

A virtual machine (VM) allows the use of another operating system (OS), e.g., Windows or macOS. Ubuntu is the preferred operating system for using ROS2. VM is easier to implement, but it will be laggy. The steps are:

- Install VirtualBox (VM).

<https://www.virtualbox.org/>

- Download Ubuntu **24.04.2 Noble** (Desktop image).
<https://releases.ubuntu.com/noble/>
- Follow the steps indicated in the next link: launching Ubuntu with VM.
<https://youtu.be/QXdFTEPXJ4M?si=zs9JWaK1tWlyxKJP>
- Some of the parameters in VM should be modified:
 - If there is a black screen at startup, change the display settings as follows:

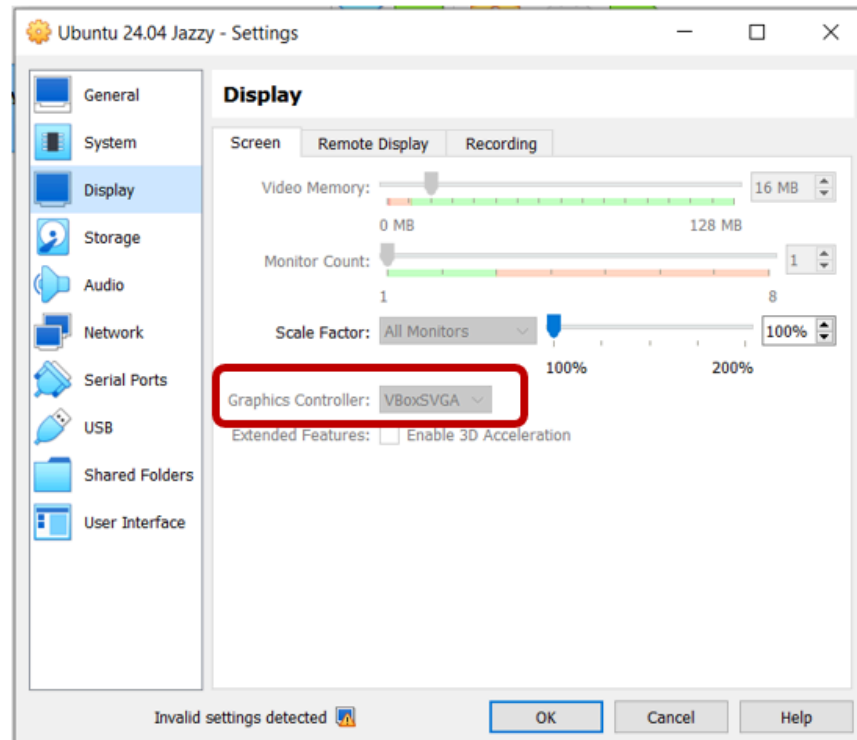


Figure 1: Display → Graphics Controller → VBoxSVGA.

- For the WiFi to work, add Adapter 1 and Adapter 2 as shown:

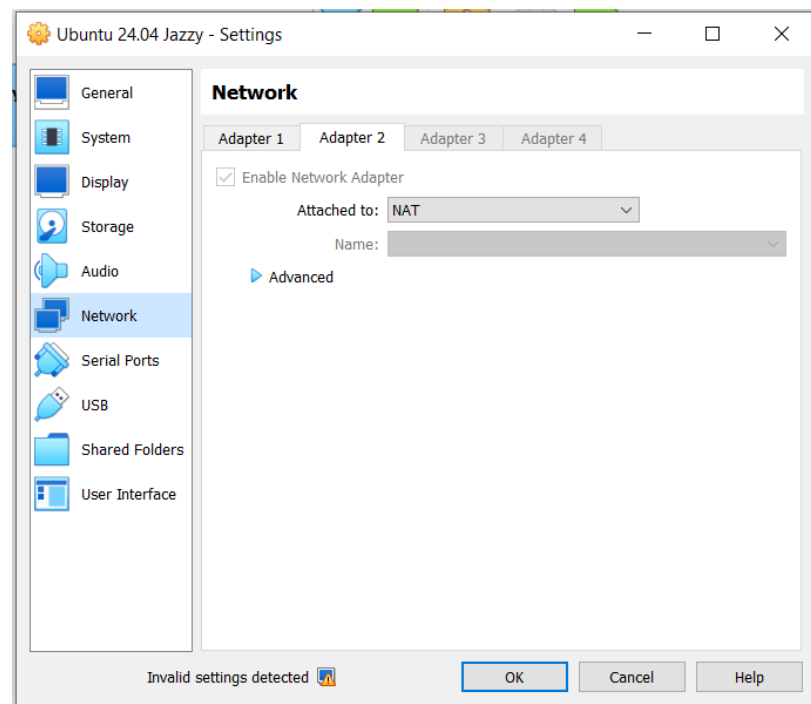
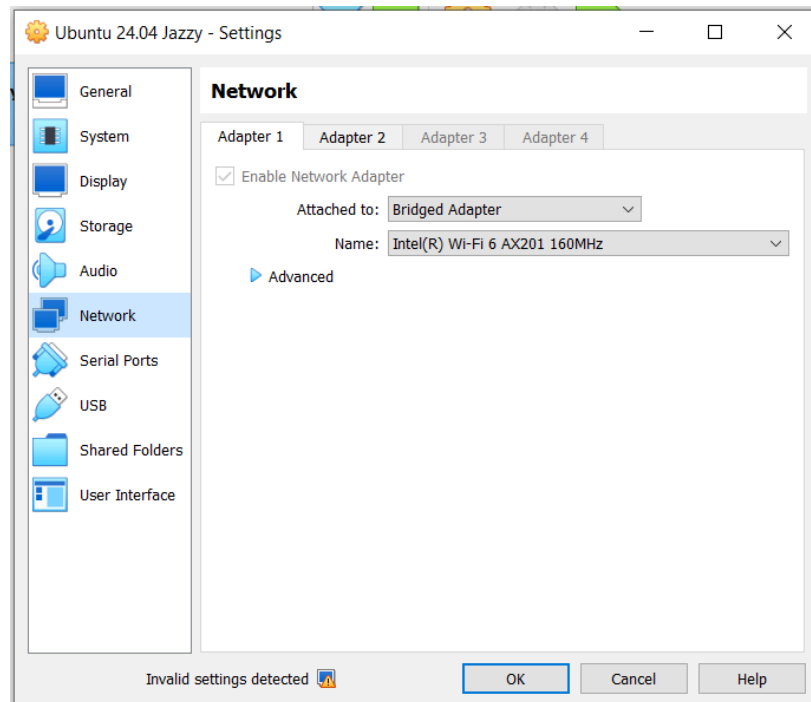


Figure 2: Network ▢ Adapter 1 ▢ Bridged Adapter

Network ▢ Adapter 2 ▢ Attached to NAT (for wired connection)

- If your computer has USB3, use it; otherwise, Ubuntu freezes while disconnecting the ESP32. For flashing purposes, the USB must be configured as follows.

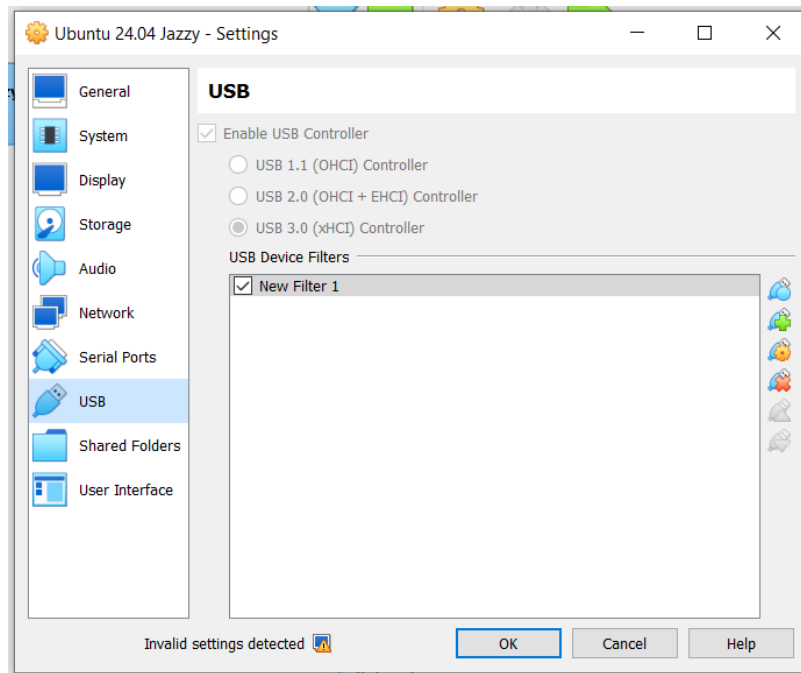


Figure 4: USB → USB 3.0 → Add New Filter

2.2. Using Ubuntu with an SSD Flash Drive

Another popular option is to dual-boot your OS computer:

- Benefit: Gazebo will be much faster
- Drawback: Ubuntu will use about 60GB on your hard drive.

A better approach is to install Ubuntu on an SSD flash drive. Please bear in mind that the following tutorial installs Ubuntu 20.04, but Ubuntu 24.04 is the preferred environment for using ROS2 Jazzy.

<https://medium.com/geekculture/installing-linux-ubuntu-20-04-on-an-external-portable-ssd-and-pitfalls-to-be-aware-of-388294e701b5>

2.3. Installing ROS2 Jazzy

The instructions are given in the next link: <https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html> . The steps are:

Tasks	Code
Open a terminal	Ctrl+Alt+T

Set locale	
Enable required repositories	
Install development tools	
Install ROS 2: Desktop Install	<pre>sudo apt update sudo apt upgrade sudo apt install ros-jazzy-desktop</pre>
Open a new terminal Copy the following code. It will load all the basic ROS2 functions at startup	<pre>echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc</pre>

2.4. Installing additional software

The following software is mandatory:

Tasks	Code
Open a terminal	Ctrl+Alt+T
Install Terminator	sudo apt install terminator
Install VSCode	sudo snap install code --classic
VSCode, install the following add-ons	PlatformIO: Best extension for programming microcontrollers properly

3. Preparing the RPI

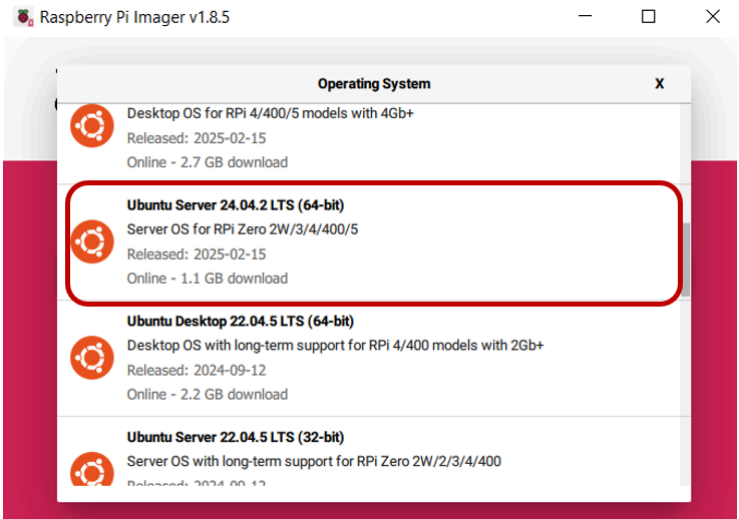
The Raspberry Pi (RPI) is a single-board computer (SBC). It is compact, lightweight, and convenient for mobile robots. It can handle Wi-Fi for communication and an Operating System (OS), which is required to use peripherals such as a camera, Lidar, microphone, and speakers. This will not be possible with a microcontroller. Of course, the robot can use a laptop instead of an SBC: it is more powerful but more expensive, bulkier, and more dangerous (what would happen if the laptop were to drop?). The procedure is similar to the preparation of the PC. However, **Ubuntu server** will be installed, as the main communication with the RPI will be SSH (remote control).

3.1. Using Raspberry Pi Imager and installing Ubuntu on the RPI

Follow the next tasks:

Tasks	Code
Install Raspberry Pi Imager	https://www.raspberrypi.com/software/
Select the RPI 5 platform Select Ubuntu server 24.04	
In the tab "General"	

Configuration of the network:	
Set hostname	E.g., alpha-bot.local, beta-bot.local, gamma-bot.local, etc.
Set username	E.g., student
Set password	E.g., student
Configure wireless LAN	SSID: e.g., name of your local hotspot (for example: from your phone) Password: e.g., the password of your local hotspot
In the tab "Services":	Enable SSH. Use password authentication
The SD card is ready to be flashed.	



Please install Ubuntu Server 24.04.2 LTS (64-bit)

3.2. Installing ROS2 Jazzy on the RPI

ROS2 will be installed on the RPI through SSH. Make sure the PC and RPI are connected to the same WiFi network, i.e., the one defined in the wireless LAN configuration with RPI Imager. As soon as the RPI is connected to your network, it will blink several times because of the required updates. This might take about 15 minutes. Then, follow the instructions:

Tasks	Code
Remote control of your RPI5	ssh id@ipaddr
For the first connection, you will need to enter the numerical ip address, which can be found with your cellphone's mobile hotspot	ssh student@alpha-robot.local ssh student@192.168.255.1
Enter your password	

Please wait a bit while the Ubuntu of your RPI is updating its core elements. It may take up to 15 minutes. The RPI may reboot itself several times.	
Update and upgrade your packages	<pre>sudo apt update</pre> <pre>sudo apt upgrade</pre>
Install Avahi for discovering locally your RPI (not installed by default with Ubuntu server)	<pre>sudo apt install avahi-daemon avahi-utils libnss-mdns -y</pre>
Reboot your RPI	<pre>sudo reboot</pre>

Install ROS2 Jazzy. The instructions are given in the next link:

<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>

Tasks	Code
Open a terminal	Ctrl+Alt+T
Set locale	
Enable required repositories	
Install development tools	
Install ROS 2: ROS-Base Install	<pre>sudo apt update</pre> <pre>sudo apt upgrade</pre> <pre>sudo apt install ros-jazzy-ros-base</pre>
Install some examples	<pre>sudo apt install ros-jazzy-demo-nodes-cpp ros-jazzy-demo-nodes-py</pre>
Open a new terminal	<pre>echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc</pre>
Copy the following code. It will load all the basic ROS2 functions at startup	

4. SSH via terminal: Testing the communication between RPI and PC

Now that ROS2 has been installed on both platforms, communication will be tested as follows:

- The RPI starts a node and publishes a topic.
- The PC listens to the topic published on the network by RPI.

Please follow the instructions on the RPI:

Tasks	Code
Open a terminal	Ctrl+Alt+T
Remote control	<pre>student@alpha-robot.local</pre>
RPI runs a ROS2 node named "publisher":	<pre>ros2 run demo_nodes_cpp talker</pre>
RPI should publish Hello World, with a number increment.	

Please follow the instructions on the PC:

Tasks	Code
Open a terminal	Ctrl+Alt+T
PC runs a ROS2 node named "listener":	ros2 run demo_nodes_cpp listener
The new terminal (PC) should show:	I heard: Hello World, with the same number increment of the RPI.

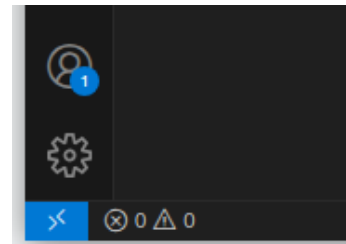
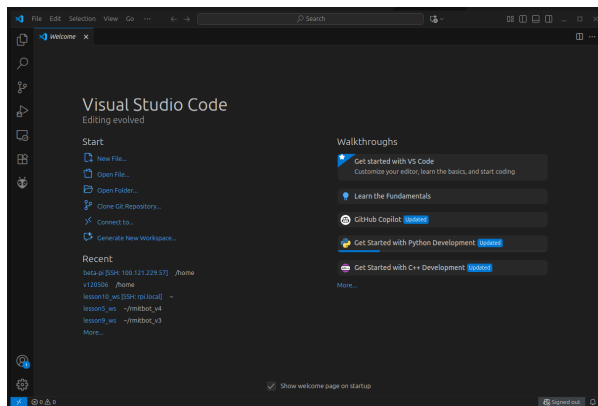
Congratulations: PC and RPI are ready to work together. Use the keyboard command Ctrl+C to kill nodes from the RPI and PC.

Use the following function to manipulate the RPI remotely:

Tasks	Code
Reboot the RPI	sudo reboot
End SSH connection	exit
Turn off RPI	sudo poweroff

5. SSH via VSCode

In Ubuntu, you should use VSCode as a preferred IDE. You may connect through SSH with VSCode as well. Code contained in the RPI can be modified more easily instead of using the app called nano in the terminal.



In the bottom left of VSCode, the SSH connection can be enabled.

This will be essential to control the robot and launch nodes remotely.

6. Going further

You now have the basic setup to use ROS2 with your PC and with the RPI. However, some of the problems are as follows:

- How to control the robot over the internet?

- How to visualize the robot publicly?

The keywords to answer these questions are:

- Tailscale,
- Zenoh,
- Digital Ocean - Droplet

ROS2 Development Tools

Table of Contents

- [1. Introduction](#)
- [2. Create a workspace, package, and launch file](#)
 - [2.1. Creating a workspace and a package](#)
 - [2.2. Creating a launch file](#)
- [3. GitHub](#)
 - [3.1. Make your first commit on GitHub](#)
 - [3.2. Clone the code from GitHub to your computer](#)
- [4. Command Line Interface \(CLI\)](#)
 - [4.1. Nodes](#)
 - [4.2. Topics](#)
 - [4.3. Services](#)
 - [4.4. Parameters](#)
 - [4.5. Actions](#)
- [5. Plotjuggler for data visualization](#)

1. Introduction

Several additional tools must be understood, such as:

- ROS2 tools: package and node creation
- Github
- Plotjuggler to visualize data

2. Create a workspace, package, and launch file

You may want to create your own package and nodes. This section gives the guidelines to do so.

2.1. Creating a workspace and a package

Task	Code
Open a terminal	Ctrl+Alt+T
Install the compiler colcon	sudo apt install python3-colcon-common-extensions
Make a directory (folder) called lesson1_ws, with a folder called src	mkdir -p lesson1_ws/src
Change directory to lesson1_ws/src	cd lesson1_ws/src
build a new package (cpp)	ros2 pkg create --build-type ament_cmake my_package

build a new package (python)	ros2 pkg create --build-type ament_python my_package
Change directory to lesson1_ws	cd ~/lesson1_ws
Build the workspace	colcon build
Source the workspace	source install/setup.bash

2.2. Creating a launch file

Tutorial: https://youtu.be/xJ3WAs8GndA?si=W_VEqCKo5m_yVBqb

A launch file allows you to launch several nodes with their associated parameters.

Task	Code
Change directory to lesson1_ws/src/my_package	cd lesson1_ws/src/my_package
Create a launch folder	mkdir launch
Change directory to the launch folder	cd launch
Create a launch file demo.launch.py	touch demo.launch.py
Come to layer back	cd ../../
Open VSCode	code .
Build the workspace	colcon build
Source the workspace	source install/setup.bash
Launch the launch file	ros2 launch my_package demo.launch.py

3. GitHub

GitHub is a platform to share code. You will use many packages available, but you will have to upload your packages as well.

3.1. Make your first commit on GitHub

Make sure you have installed Git (please refer to the setup of your dev computer). Create a new repository on your GitHub account – for example, lesson1_ws. Do not add a README file yet.

On VSCode, go to your folder containing the code and additional folders:

Task	Code
Open a terminal in VSCode	
Initialization	git init
Add everything	git add .
	git commit -m "first commit"
	git branch -M main

Make a commit Create a branch Remote path (change the address to the repository you have created):	git remote add origin https://github.com/ROS2-AutoBot/lesson1_ws.git
Sometimes, there is already an origin, check with	git remote -v
You may change it with the following cli	git remote set-url origin https://github.com/ROS2-AutoBot/lesson1_ws.git
Push it to GitHub	git push -u origin main
Normally, the entire folder should be copied on your online repository	

3.2. Clone the code from GitHub to your computer

This is an essential task: since most of the code has been developed already, the aim is to clone the correct code and adapt it to your needs. In a terminal, navigate to the folder/workspace where you would like to clone the repository.

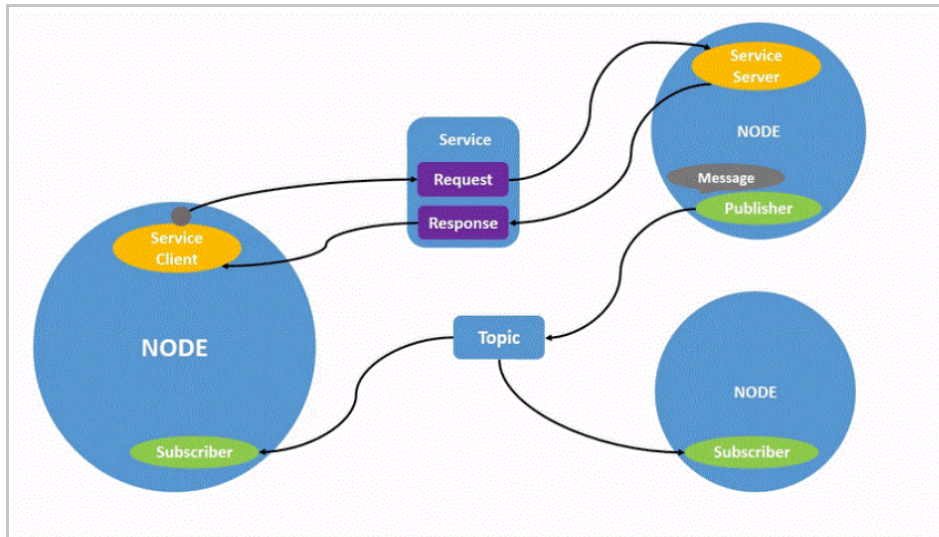
Task	Code
Open a terminal	
Change directory	cd <workspace>
Clone the repository	git clone https://github.com/ROS2-AutoBot/lesson1_ws.git
Change back to the workspace directory, compile the project, and source the workspace	cd <workspace> colcon build source install/setup.bash
If there is a compilation error, you may have to remove build, install, log, and colcon build.	

4. Command Line Interface (CLI)

It will be essential to debug in ROS2. Here are the base tools for debugging.

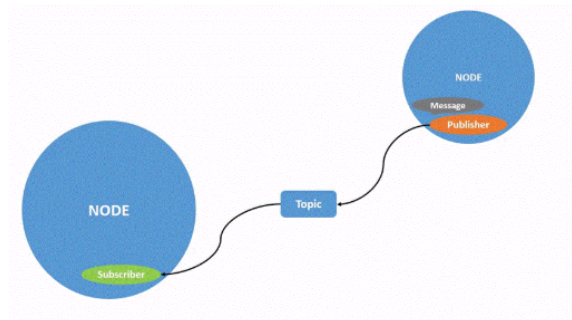
4.1. Nodes

Each node in ROS should be responsible for a single, modular purpose, e.g., controlling the wheel motors or publishing the sensor data from a laser rangefinder. Each node can send and receive data from other nodes via topics, services, actions, or parameters.



Task	Code
Launch a node (piece of code)	<pre>ros2 run <package_name> <executable_name> ros2 run turtlesim turtlesim_node ros2 run turtlesim turtle_teleop_key</pre>
Show you the names of all running nodes	<pre>ros2 node list</pre>
Remapping: reassign default node properties (e.g. name)	<pre>ros2 run turtlesim turtlesim_node --ros-args --remap __node:=my_turtle</pre>
Access more information	<pre>ros2 node info <node_name> ros2 node info /my_turtle</pre>

4.2. Topics

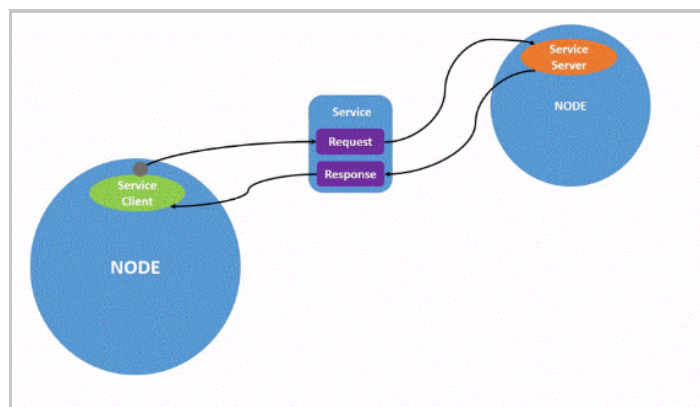


Task	Code
------	------

Graphical interface: shows node and topics	rqt_graph
List of the topic, with their message type	ros2 topic list ros2 topic list -t
See data being published in a topic	ros2 topic echo <topic_name> ros2 topic echo /turtle1/cmd_vel
list the number of publisher and subscriber on the topic	ros2 topic info <topic_name> ros2 topic info /turtle1/cmd_vel
learn details about a message type	ros2 interface show <msg_type> ros2 interface show geometry_msgs/msg/Twist
publish manually on the topic from the terminal	ros2 topic pub <topic_name> <msg_type> '<args>' ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}" ros2 topic pub --once -w 2 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
Frequency and bandwidth of topic	ros2 topic hz /turtle1/pose ros2 topic bw /turtle1/pose
List a list of available topics of a given type	ros2 topic find <topic_type> ros2 topic find geometry_msgs/msg/Twist

4.3. Services

Services are based on a call-and-response model versus the publisher-subscriber model of topics. While topics allow nodes to subscribe to data streams and get continual updates, services only provide data when they are specifically called by a client.



Task	Code
return a list of all the services currently active	ros2 service list ros2 service list -t
To find out the type of a service	ros2 service type <service_name> ros2 service type /clear
To see information of a particular service	ros2 service info <service_name> ros2 service info /clear
to find all the services of a specific type	ros2 service find <type_name> ros2 service find std_srvs/srv/Empty
to know the structure of the input arguments	ros2 interface show <type_name> ros2 interface show std_srvs/srv/Empty ros2 interface show turtlesim/srv/Spawn
can call a service	ros2 service call <service_name> <service_type> <arguments> ros2 service call /clear std_srvs/srv/Empty ros2 service call /spawn turtlesim/srv/Spawn "{x: 2, y: 2, theta: 0.2, name: ''}"
To see the data communication between a service client and a service server	- ros2 service echo <service_name service_type> <arguments> - ros2 launch demo_nodes_cpp introspect_services_launch.py - ros2 param set /introspection_service service_configure_introspection contents - ros2 param set /introspection_client client_configure_introspection contents
List a list of available topics of a given type	ros2 topic find <topic_type> ros2 topic find geometry_msgs/msg/Twist

4.4. Parameters

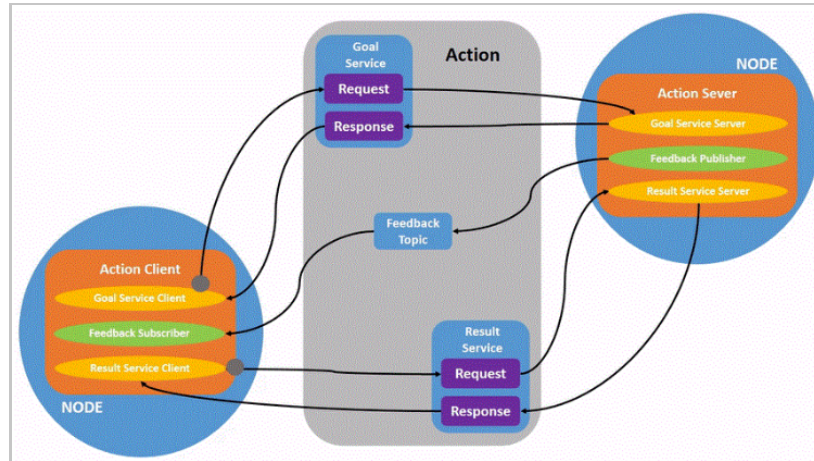
A parameter is a configuration value of a node. You can think of parameters as node settings.

Task	Code
To see the parameters belonging to your nodes	ros2 param list
To display the type and current value of a parameter	ros2 param get <node_name> <parameter_name> ros2 param get /turtlesim background_g
To change a parameter's value at runtime	ros2 param set <node_name> <parameter_name> <value> ros2 param set /turtlesim background_r 150
view all of a node's current parameters	ros2 param dump <node_name> ros2 param dump /turtlesim > turtlesim.yaml
load parameters from a file	ros2 param load <node_name> <parameter_file>

	ros2 param load /turtlesim turtlesim.yaml
To start the same node using your saved parameter values	<pre>ros2 run <package_name> <executable_name> --ros-args --params-file <file_name></pre> <pre>ros2 run turtlesim turtlesim_node --ros-args --params-file turtlesim.yaml</pre>

4.5. Actions

They consist of three parts: a goal, feedback, and a result. Their functionality is similar to services, except that actions can be canceled. They also provide steady feedback, as opposed to services that return a single response.



Task	Code
Node information (sub, pub, service, etc.)	<pre>ros2 node info /turtlesim</pre> <pre>ros2 node info /teleop_turtle</pre>
To identify all the actions in the ROS graph	<pre>ros2 action list</pre> <pre>ros2 action list -t</pre>
to check the action type for the action	<pre>ros2 action type /turtle1/rotate_absolute</pre>
further introspect	<pre>ros2 action info /turtle1/rotate_absolute</pre>
the structure of the action type	<pre>ros2 interface show turtlesim/action/RotateAbsolute</pre>
send an action goal with feedback	<pre>ros2 action send_goal <action_name> <action_type> <values></pre> <pre>ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}"</pre> <pre>ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: -1.57}" --feedback</pre>

5. Plotjuggler for data visualization

Information: <https://index.ros.org/p/plotjuggler/>

Tutorial: <https://youtu.be/MnMGjvYxlUk?si=3zabX6mJ8EmFIR0X>

Install ros package: `sudo apt install ros-jazzy-plotjuggler-ros`

Launch plotjuggler: `ros2 run plotjuggler plotjuggler`

Task	Code
Install ros package Launch plotjuggler	<code>sudo apt install ros-jazzy-plotjuggler-ros</code> <code>ros2 run plotjuggler plotjuggler</code>
In a new terminal launch the teleoperation keyboard node	<code>ros2 run teleop_twist_keyboard teleop_twist_keyboard</code>
In a new terminal Check the node <code>/plotjuggler</code> and <code>/teleop_twist_keyboard</code>	<code>ros2 node list</code>
In a new terminal Check the topic <code>/cmd_vel</code> , <code>/parameter_events</code> , <code>/rosout</code>	<code>ros2 topic list</code>
In plotjuggler, choose the correct topic to stream and to visualize the data	

Plotjuggler will be essential to record and measure the robot movement and include it in your final report.

ROS2 in Simulation (Gazebo)

Table of Contents

[1. Introduction](#)

1. Introduction

Simulation in ROS2 allows one to get comfortable with the concept of:

- Nodes, topics, packages
- Debugging in ROS2

Gazebo is used as a physics engine. Unity and Unreal Engine are both popular 3D physics engines. They both work with ROS2 as well, but require more coding experience. Gazebo has been developed in parallel with ROS2 to avoid robot developers spending too much time coding like video game developers. This session presents the following packages:

- Description: urdf
- Controller: ros2_control
- Localization: robot_localization
- Mapping: slamtoolbox
- Navigation: nav2
- Vision: apriltag

Robot Description

Table of Contents

[1. Introduction](#)

[2. Tasks](#)

[3. Explanations](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Introduction

The prerequisites are:

- Ubuntu 24.04 Noble runs on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This section is in simulation only. The hardware is not required.

This section presents how to get started with ROS2 and Gazebo. The URDF defines the physical structure of a robot. The main packages used are:

- robot_state_publisher: a node that publishes the structure of the robot
- Rviz: a node that is used for topic visualization
- Gazebo: the physics engine, i.e., the virtual model of the robot
- joint_state_publisher_gui: a node to move the wheel of the robot

2. Tasks

This first package is called description, and it introduces the concept of URDF: this is the description of your robot in the ROS2 environment in an XML format. Moreover, the package includes Gazebo, a physics engine, and RViz, a topic visualizer. The table is given as an indication for the command lines in the terminal. It is recommended to enter them one by one.

Tasks	Code
Install gazebo	sudo apt-get install ros-jazzy-ros-gz
Install rviz2	normally, rviz2 is already installed with ROS2 jazzy desktop
Clone the repo from GitHub	git clone https://github.com/ROS2-rmitbot/lesson1_ws.git
Navigate to the workspace	cd ~/lesson1_ws
Remove the folders build install log	rm -rf build install log
Build the packages	colcon build --symlink-install
Source the workspace	source install/setup.bash
Open VSCode	code .
In a terminal, launch RViz and Gazebo	ros2 launch rmitbot_description rsp.launch.py

	ros2 launch rmitbot_description jsp_gui.launch.py ros2 launch rmitbot_description rviz.launch.py ros2 launch rmitbot_description gazebo.launch.py
Alternatively, launch the bringup file	ros2 launch rmitbot_bringup rmitbot.launch.py
Look at the topic list	ros2 topic list
Look at the node list	ros2 node list
Look at the node/topic graph	ros2 run rqt_graph rqt_graph

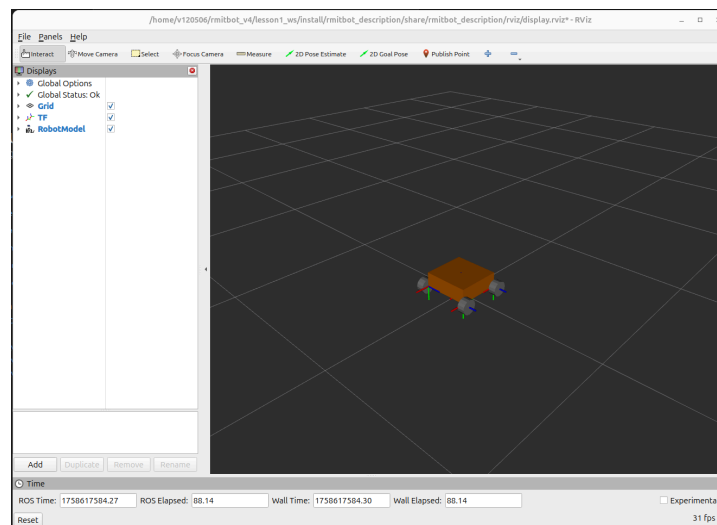
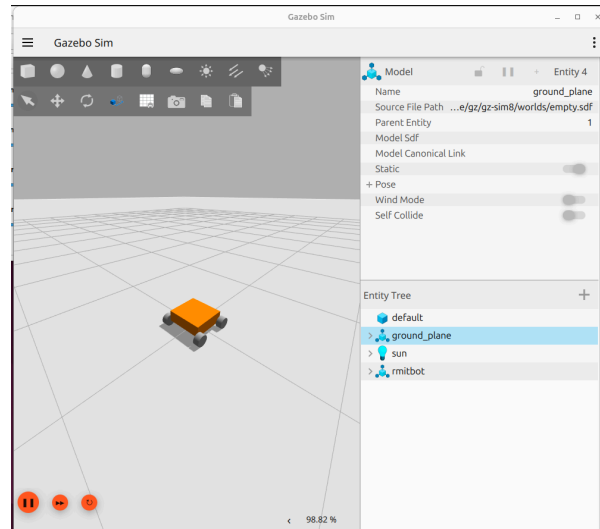


Figure 1: Gazebo is useful when the hardware is not available. Rviz2 is a graphical interface to visualize topics.

3. Explanations

This may be the first time you are using Ubuntu and ROS 2. The learning curve is steep, but rewarding in the long run. General knowledge about Ubuntu and ROS2 is:

- sudo: it means “superuser do ...”. Namely, launch a program with admin rights.
- ros2 launch ...: it means “in the ros2 framework, launch a specific program”.
- Students are invited to use resources to gain an understanding of the provided code.

This package is called rmitbot_description. It describes the robot and its environment. By now, you should understand that the commands are started from the terminal, which is not as simple as pointing and clicking at an icon in a folder. A launch file is helpful to launch several commands at the same time with different parameters. For instance, the bringup launch file starts the following packages:

- robot_state_publisher: launch the URDF file.
- Display: this starts RViz
- Gazebo: This is the physics engine of ROS2.
- joint_state_publisher_gui: a node to move the wheel of the robot

The robot can be visualized in RViz and Gazebo, but it cannot move. This is covered in the next package, rmitbot_controller. Please look at the code located in the src folder to gain insight into the ROS2 folder architecture.

- The folder **launch** contains the launch file.
- The folder **meshes** contains STL files.
- The folder **RViz** contains a configuration file for the RViz node.
- The folder **URDF** contains the description of the robot in an XML format.
- The **CMakeLists.txt** is used for the compilation of the package. The folder launch, URDF, RViz, and meshes must be added to the path for compilation.
- The **package.xml** is used for compilation of the package and must contain the executable dependencies: robot_state_publisher, joint_state_publisher_gui, rviz, ros2launch, and ros_gz_sim for Gazebo.

RViz and Gazebo look like similar tools, but they have different purposes:

- RViz is just for visualization: it subscribes to topics, such as robot frames (TF), and later on, sensors can be visualized (IMU, Lidar, encoders).
- Gazebo is for simulation: Usually, it is used at the early stage to make sure that all the nodes for communication are working, and then the code can be deployed to the real robot. The encoder, the imu, the lidar, and even the camera and depth camera can be simulated in Gazebo through plugins.

4. Packages, nodes, and topics

These are a summary of the packages, nodes and topics involved in this first part:

Packages	File	Description
rmitbot_bringup	bringup.launch.py	Launch the other packages
rmitbot_description	rsp.launch.py	robot_state_publisher
	rviz.launch.py	Rviz
	gazebo.launch.py	Gazebo
	jsp_gui.launch.py	joint_state_publisher_gui

Package/node	Topic	Description
ROS2	/parameter_events	For ROS2 to function
	/rosout	
rviz2	/clicked_point	Click point in RViz
	/goal_pose	Target pose of the robot (for nav2)
	/initialpose	Initial pose of the robot (for slam_toolbox)
gazebo	/clock	Sim time for node synchronization
joint_state_publisher_gui	/joint_states	Position, velocity, and effort of robot joints. Normally, it is given by the robot's hardware (encoder), but for the moment, joint_state_publisher is used to fake it.
robot_state_publisher	/robot_description	Description of the robot in XML format (also called URDF)
	/tf	Dynamic tf (odom --> base_footprint)
	/tf_static	Static tf (base_footprint --> rest of the robot)

5. Future works

The URDF of the robot is quite simple: it is based on cubes and cylinders. One of the tasks is to model your robot on CAD software and transfer it to the URDF format.

Robot Controller

Table of Contents

[1. Introduction](#)

[2. Tasks](#)

[3. Explanations](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Introduction

The prerequisites are:

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This section is in simulation only. The hardware is not required.

The `ros2_control` and `ros2_controllers` packages are useful to control the robot, whether it is a differential robot, a mecanum, or a robot arm. It deals with the inverse kinematics and the command to be sent to the hardware/simulation.

2. Tasks

Previously, the robot could be visualized in RViz and Gazebo, but could not move. This package `rmitbot_controller` introduces the controller manager. It loads the following concepts:

- Controller manager: the core of the package. It supervises the input/output of the robot.
- Joint_state_broadcaster_spawner: part of the controller manager. It starts the node to read the state of the robot (wheel position and velocity)
- Controller_spawner: part of the controller manager. It starts the node to translate the Cartesian velocity command to the wheel velocity command.
- teleopkeyboard: The keyboard can be used to control the robot in the Cartesian space.

Tasks	Code
Install <code>ros2_control</code> and <code>ros2_controller</code>	<code>sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers</code>
Install <code>gz_ros2_control</code>	<code>sudo apt-get install ros-jazzy-gz-ros2-control</code>
Install teleopkeyboard	<code>sudo apt install ros-jazzy-teleop-twist-keyboard</code>
Install an external terminal	<code>sudo apt install xterm</code>
Clone the repo from GitHub	<code>git clone https://github.com/ROS2-rmitbot/lesson2_ws.git</code>
Navigate to the workspace	<code>cd ~/lesson2_ws</code>
Remove the folders build install log	<code>rm -rf build install log</code>
Build the packages	<code>colcon build</code>

Source the workspace	source install/setup.bash
Open VSCode	code .
Launch rsp	ros2 launch rmitbot_description rsp.launch.py
Launch RViz,	ros2 launch rmitbot_description rviz.launch.py
Launch Gazebo,	ros2 launch rmitbot_description gazebo.launch.py
Launch controller,	ros2 launch rmitbot_controller controller.launch.py
Launch teleopkeyboard	ros2 launch rmitbot_controller teleopkeyboard.launch.py
Alternatively, launch the bringup file	ros2 launch rmitbot_bringup rmitbot.launch.py
Check the publisher/subscriber of a topic	ros2 topic info -v <yourtopic> e.g., ros2 topic info -v /rmitbot_controller/cmd_vel
Use the command line from the previous lesson	ros2 node list ros2 topic list ros2 run rqt_graph rqt_graph
Install plotjuggler	sudo apt install ros-jazzy-plotjuggler ros-jazzy-plotjuggler-ros
Run plotjuggler	ros2 run plotjuggler plotjuggler

```

teleop_twist_keyboard

Moving around:
  u    i    o
  j    k    l
  m    ,    .

For Holonomic mode (strafing), hold down the shift key:
-----
  U    I    O
  J    K    L
  M    <    >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently:   speed 0.5      turn 1.0
  
```

Figure 1: The teleopkeyboard.

3. Explanations

The package `ros2_control` is used to move the robot in the simulated environment with the keyboard. Later, we will see how to go from simulation to real hardware, but for the moment, you should get comfortable with the concept of nodes and topics. The parameter `use_sim_time` has been set to `true` for all nodes since Gazebo is used (simulation environment). When real hardware is used, then `use_sime_time` will be set to `false`. The packages have been modified as follows:

- `rmitbot_description`:

- o **urdf/rmitbot_ros2_control.xacro**: This file describes which part of the robot is controlled and which sensors send data to the computer. It contains: command_interface, state_interface, and gazebo plugins.
- o launch/display.launch.py: the joint state publisher GUI is disabled. We no longer need it as ros2_control is used to publish the joint_state (position and velocity).
- rmitbot_controller
 - o **config/rmitbot_controller.yaml**: configuration of the ros2_controller package
 - o joint_state_broadcaster: this node is broadcasting /joint_state, which is essential for robot_state_publisher, which broadcasts the tf of the robot.
 - o rmitbot_controller: it loads the node diff_drive_base_controller. This node publishes the topic /rmitbot_controller/odom, which is the main way of getting the pose of the robot from encoders. It subscribes to a twist /rmitbot_controller/cmd_vel and performs the inverse kinematics to send the command to the motors of the robot.
 - o teleopkeyboard: Use of keyboard to control the robot in Gazebo.
 - o The controller package is handling joint_state_broadcaster and robot_state_publisher. That would be important later while using the hardware.

4. Packages, nodes, and topics

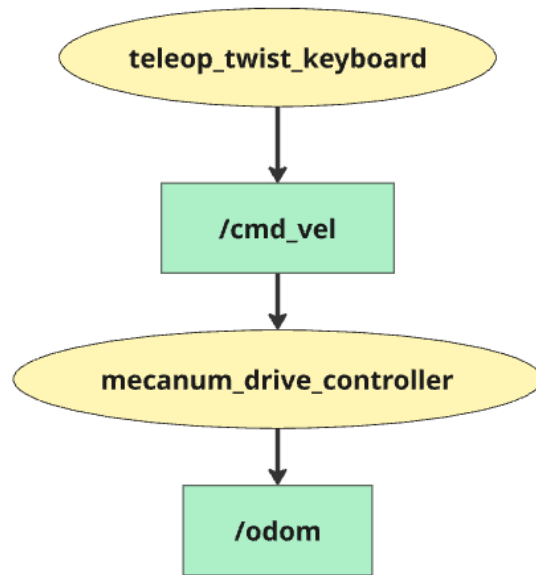
Packages	Files	Description
rmitbot_bringup	bringup.launch.py	Launch the other packages
rmitbot_description	rsp.launch.py	robot_state_publisher
	rviz.launch.py	Rviz
	gazebo.launch.py	Gazebo
	jsp_gui.launch.py	Joint_state_publisher_gui (not used anymore)
rmitbot_controller	controller.launch.py	Controller manager: jsb spawner, controller spawner
	teleopkeyboard.launch.py	Launch the input/output of the robot.

Package	Topic	Description
ROS2 standard	/parameter_events	n/a
	/rosout	n/a
rviz	/clicked_point	Click point in RViz
	/goal_pose	Target pose of the robot (for nav2)
	/initialpose	Initial pose of the robot (for slam_toolbox)

gazebo	/clock	Sim time for node synchronization
robot_state_publisher	/robot_description /tf /tf_static	Description of the robot in XML format Dynamic tf (odom --> base_footprint) Static tf (base_footprint --> rest of the robot)
controller_manager	/controller_manager/activity /controller_manager/introspection_data/full /controller_manager/introspection_data/names /controller_manager/introspection_data/values	For debugging only
ros2_controllers	/joint_states /rmitbot_controller/cmd_vel /rmitbot_controller/odom /rmitbot_controller/cmd_vel_out /joint_state_broadcaster/transition_event /dynamic_joint_states /rmitbot_controller/transition_event	Important: pos, vel, effort of the joint Important: Cartesian vel command Important: odometry (pose of the robot)

5. Future works

- The simulated robot can be controlled with the keyboard, which is fine for debugging. The use of a joystick would be appreciated.
- The simulated robot can move, but the speed is unknown. One can use the tool plotjuggler to visualize and record important states, e.g., wheel velocities
- Important topics are as follows: can you define them and visualize them with RQT, RViz, or PlotJuggler?
 - /cmd_vel
 - /odom
 - /joint_states
 - /controller_state
 - /robot_description



Robot Localization

Table of Contents

[1. Introduction](#)

[2. Tasks](#)

[3. Explanations](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Introduction

The prerequisites are:

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is a simulation only. The hardware is not required.

The package `robot_localization` introduces a famous algorithm: the Kalman filter. It fuses the data between the wheel encoder and IMU to obtain the orientation of the robot.

2. Tasks

The localization of a robot is one of the hardest challenges in mobile robots. The encoder gives the pose (position and orientation) of a robot, but it can drift due to slippage of the wheel on the floor, inaccuracy of the encoders, dimension error of the robot geometry, etc. An IMU is a valuable addition to the system, as it is not affected by slippage. However, the IMU is sensitive to noise and vibration; thus, an average of both sensors would be necessary: this is the purpose of the `robot_localization` package and its Kalman filter.

Tasks	Code
Install localization	<code>sudo apt install ros-jazzy-robot-localization</code>
Install transform tools	<code>sudo apt install ros-jazzy-tf2-tools</code>
Clone the repo from GitHub	<code>git clone https://github.com/ROS2-rmitbot/lesson3_ws.git</code>
Navigate to the workspace	<code>cd ~/lesson3_ws</code>
Remove the folders build install log	<code>rm -rf build install log</code>
Build the packages	<code>colcon build</code>
Source the workspace	<code>source install/setup.bash</code>
Open VSCode	<code>code .</code>
Launch rsp	<code>ros2 launch rmitbot_description rsp.launch.py</code>
Launch RViz,	<code>ros2 launch rmitbot_description rviz.launch.py</code>
Launch Gazebo,	<code>ros2 launch rmitbot_description gazebo.launch.py</code>

Launch controller,	<code>ros2 launch rmitbot_controller controller.launch.py</code>
Launch teleopkeyboard	<code>ros2 launch rmitbot_controller teleopkeyboard.launch.py</code>
Launch local localization	<code>ros2 launch rmitbot_localization local_localization.launch.py</code>
Alternatively, launch the bringup launch	<code>ros2 launch rmitbot_bringup rmitbot.launch.py</code>
Visualize the tf tree	<code>ros2 run tf2_tools view_frames</code> <code>evince <frame.pdf></code>

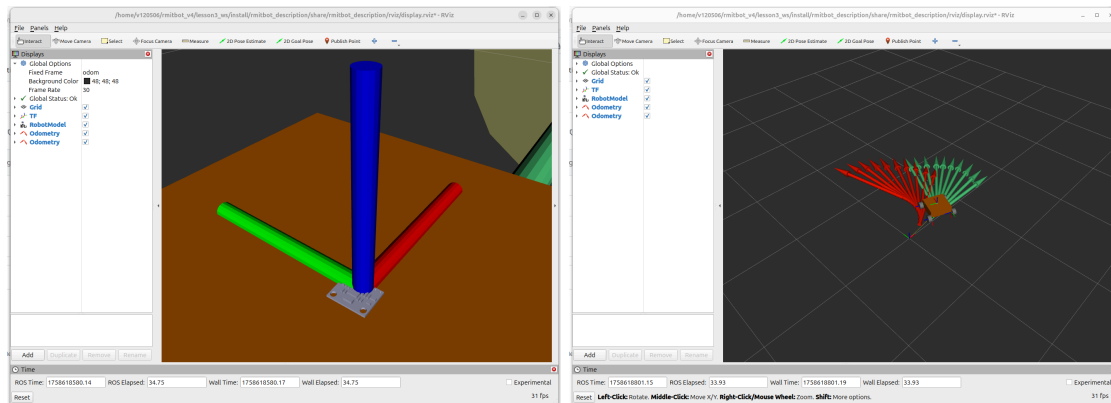


Figure 1: The IMU is located on top of the robot. Its addition is beneficial as the robot slips quite often during rotation.

There are two important frames, which are `odom` and `base_footprint`. They respectively correspond to the initial pose of the robot and its current pose. Initially, the transform (tf) between `odom` and `base_footprint` is broadcast by the `ros2_control` package (obtained by the encoders). The localization package outputs a better estimate since it uses the fusion between imu and encoders. Thus, the parameter “`enable_odom_tf`” in the controller YAML file must be set to false.

3. Explanations

The pose estimate can come from different sensors, namely:

- encoders: also called odometry or dead reckoning.
- imu: inertial measurement unit
- GPS (outdoor): absolute positioning of the robot
- LiDAR odometry: LiDAR odometry estimates motion by aligning successive LiDAR scans
- Visual odometry: works better with a depth camera and is similar to lidar odometry.
- Visual Apriltag: tag are used to give absolute positioning of the robot

Tasks done in each package:

- rmitbot_description:
 - o Add a new IMU in the robot_description URDF.xacro (loading stl file graphics)
 - o Add the STL file in the meshes folder (already inside)
 - o Add gazebo.xacro information (add plugin imu, add noise for more realistic physics)
 - o Add gz_ros2_bridge for the imu (clock@rosgraph already added to use sim_time)
 - o Add dependencies in the package.xml file
- rmitbot_localization: new package
 - o ekf.yaml: defines the parameters for the extended Kalman filter (core of the robot_localization)
 - o local_localization.launch.py: launch file to start the local localization

Package modification

rmitbot_description:

- meshes/imu_link.STL added
- urdf/rmitbot_imu..xacro added
- launch/gazebo.launch.py: gz_ros2_bridge added (required for imu plugin)
- package.xml: updated to include ros_gz_bridge

rmitbot_localization:

- ekf.yaml: defines the parameters for the extended Kalman filter

4. Packages, nodes, and topics

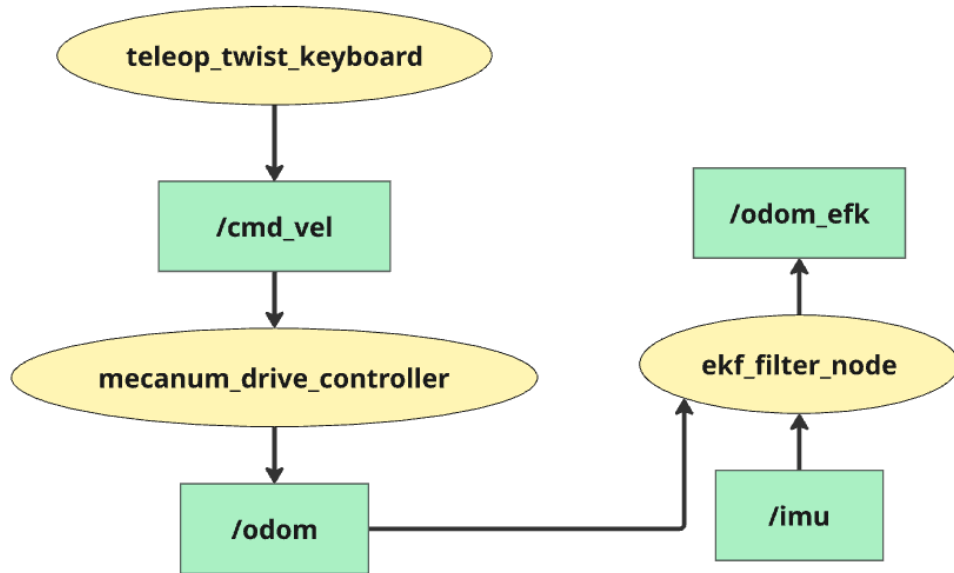
Packages	Files	Description
rmitbot_bringup	bringup.launch.py	Launch the other packages
rmitbot_description	rsp.launch.py	robot_state_publisher
	rviz.launch.py	Rviz
	gazebo.launch.py	Gazebo
	jsp_gui.launch.py	Joint_state_publisher_gui (not used anymore)
rmitbot_controller	controller.launch.py	Controller manager: jsb spawner, controller spawner
	teleopkeyboard.launch.py	Launch the input/output of the robot.
rmitbot_localization	local_localization.launch.py	Launch the sensor fusion (ekf) between wheel encoder and imu

Package	Topic	Description
ROS2 standard	/parameter_events	n/a
	/rosout	n/a
rviz	/clicked_point	Click point in rviz
	/goal_pose	Target pose of the robot (for nav2)
	/initialpose	Initial pose of the robot (for slam_toolbox)
gazebo	/clock	Sim time for node synchronization
robot_state_publisher	/robot_description	Description of the robot in xml format
	/tf	Dynamic tf (odom --> base_footprint)
	/tf_static	Static tf (base_footprint --> rest of the robot)
controller_manager	/controller_manager/activity	For debugging only
	/controller_manager/introspection_data/full	
	/controller_manager/introspection_data/names	
	/controller_manager/introspection_data/values	
ros2_controllers	/joint_states	Important: pos, vel, effort of the joint
	/rmitbot_controller/cmd_vel	Important: Cartesian vel command
	/rmitbot_controller/odom	Important: odometry (pose of the robot)
	/rmitbot_controller/cmd_vel_out	
	/joint_state_broadcaster/transition_event	
	/dynamic_joint_states	
	/rmitbot_controller/transition_event	
robot_localization	/imu/out	Imu data (actually from gazebo, but used in robot_localization)
	/odometry/filtered	Ododom from ekf: filtered and less prone to drift than /rmitbot_controller/odom

5. Future works

- The robot will drift, lose some encoder counts, and slip on some surface. Can you quantify the error in orientation with plotjuggler?
- Important topics are as follows: can you define them and visualize them with RQT, RViz, or PlotJuggler?
 - /robot_description
 - /cmd_vel
 - /odom

- /joint_states
- /controller_state
- /imu
- /odom_efk



Robot Mapping

Table of Contents

[1. Introduction](#)

[2. Tasks](#)

[3. Explanations](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Introduction

The prerequisites are:

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is simulation only. The hardware is not required.

The package slamtoolbox is one of the most precise ways to localize the robot. Odometry drifts over time, but the laser from the lidar can be trustworthy.

2. Tasks

Lidar is the most important sensor for autonomous robots and is the main transition between a simple Arduino robot to a more sophisticated platform.

Tasks	Code
Install the slam toolbox	<code>sudo apt install ros-jazzy-slam-toolbox</code>
Clone the repo from GitHub	<code>git clone https://github.com/ROS2-rmitbot/lesson4_ws.git</code>
Navigate to the workspace	<code>cd ~/lesson4_ws</code>
Remove the folders build install log	<code>rm -rf build install log</code>
Build the packages	<code>colcon build</code>
Source the workspace	<code>source install/setup.bash</code>
Open VSCode	<code>code .</code>
Launch the bringup launch	<code>ros2 launch rmitbot_bringup rmitbot.launch.py</code>

On Gazebo, it is possible to see the ray from the Lidar (Lidar Visualisation). Add some shapes in Gazebo and look at the cloud point on RViz. Then move the robot around: you should see the construction of the map. However, the lidar ray cannot be rendered while using a VM in Gazebo; however, the /scan messages are still available in RViz and can be visualized and used to do the mapping.

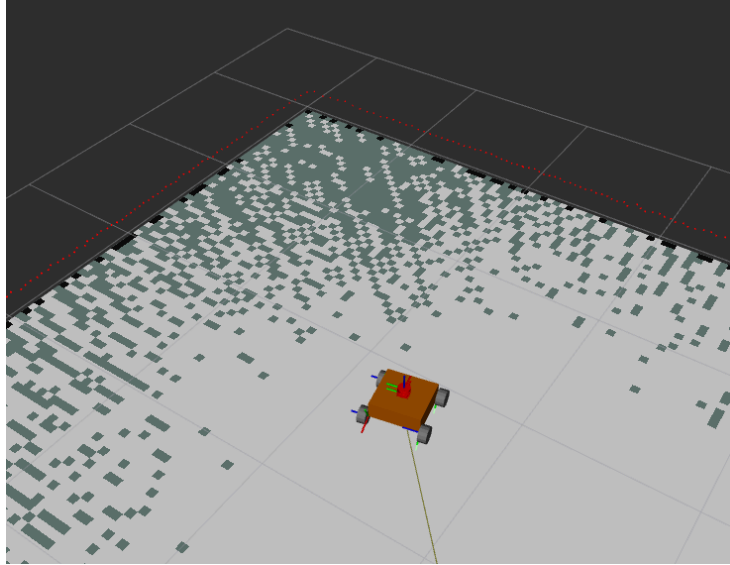


Figure 1: The robot detects the wall and maps its surroundings thanks to the Lidar and SLAM.

3. Explanations

A lidar has been added to the URDF, which includes a plugin in Gazebo. The lidar cannot construct a map if there is no obstacle, thus a simple world has been added, which consists of four walls that construct an 8m x 8m environment. Slamtoolbox constructs the map of the surroundings and corrects the positioning of the robot as well, but it requires minimum movement from the robot: the principle is similar to a blind person walking in a room and touching the side of the wall to navigate in the environment. There are now three important frames:

- odom: initial pose of the robot from the robot_localization
- base_footprint: the current pose of the robot
- map: the inertial frame. The odometry from encoders only tends to drift: you want to travel 5m with your robot, but you may achieve only 4.95m. The imu can correct the heading, but not the linear accumulated error. The lidar, coupled with the SLAM technique, allows the robot to know precisely its position relative to its environment. The map frame is the fixed frame, the odom frame is the initial pose of the robot that drifts due to accumulated error, and base_footprint is the actual pose of the robot.

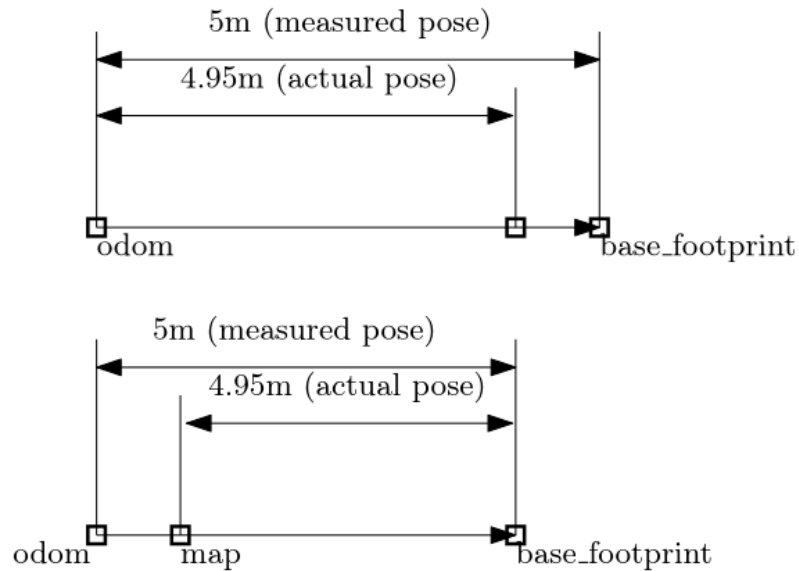


Figure 1: Comparison between odometry only and SLAM

- At the top, the robot's position is based on odometry only. The measured pose of the robot given by odometry is 5m, but the real, actual pose is 4.95m: the robot has drifted by 0.05m.
- On the bottom, the odometry still gives a measurement of 5m, but the relative position between the odom and the map is updated by the lidar. The relative position between the map and the robot is thus correct.

4. Packages, nodes, and topics

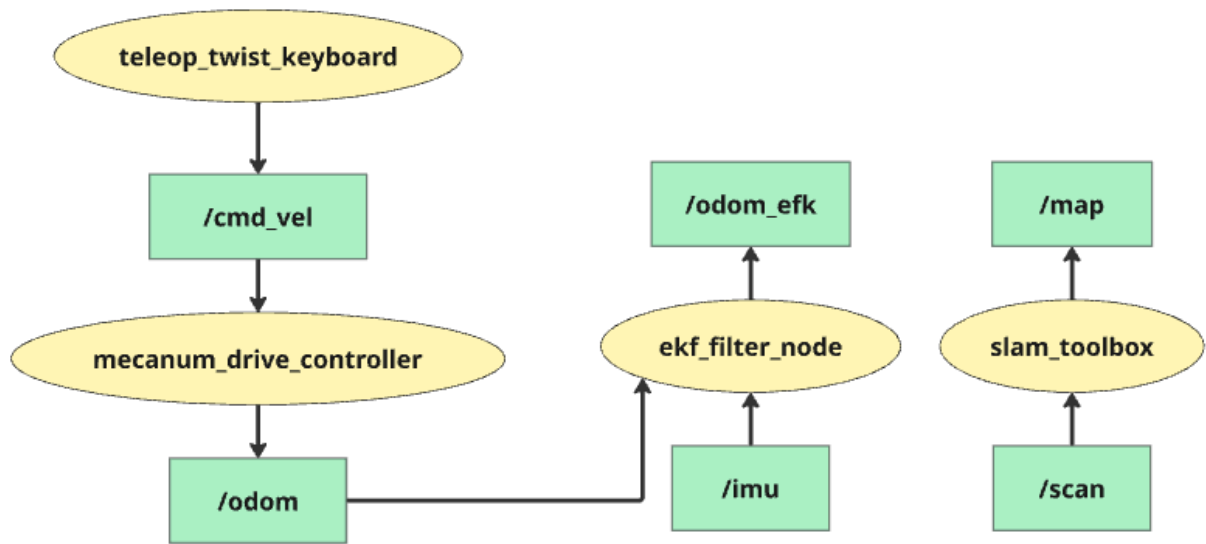
Packages	Nodes	Description
rmitbot_bringup	bringup.launch.py	Launch the other packages
rmitbot_description	rsp.launch.py	robot_state_publisher
	rviz.launch.py	Rviz
	gazebo.launch.py	Gazebo
	jsp_gui.launch.py	Joint_state_publisher_gui (not used anymore)
rmitbot_controller	controller.launch.py	Controller manager: jsb spawner, controller spawner
	teleopkeyboard.launch.py	Launch the input/output of the robot.
rmitbot_localization	local_localization.launch.py	Launch the sensor fusion (ekf) between wheel encoder and imu
rmitbot_mapping	mapping.launch.py	Launch the lidar and slam

Package	Topic	Description
ROS2 standard	/parameter_events	n/a

	/rosout	n/a
rviz	/clicked_point /goal_pose /initialpose	Click point in rviz Target pose of the robot (for nav2) Initial pose of the robot (for slam_toolbox)
gazebo	/clock	Sim time for node synchronization
robot_state_publisher	/robot_description /tf /tf_static	Description of the robot in xml format Dynamic tf (odom --> base_footprint) Static tf (base_footprint --> rest of the robot)
controller_manager	/controller_manager/activity /controller_manager/introspection_data/full /controller_manager/introspection_data/names /controller_manager/introspection_data/values	For debugging only
ros2_controllers	/joint_states /rmitbot_controller/cmd_vel /rmitbot_controller/odom /rmitbot_controller/cmd_vel_out /joint_state_broadcaster/transition_event /dynamic_joint_states /rmitbot_controller/transition_event	Important: pos, vel, effort of the joint Important: Cartesian vel command Important: odometry (pose of the robot)
robot_localization	/imu/out /odometry/filtered	Imu data (actually from gazebo, but used in robot_localization) Odom from ekf: filtered and less prone to drift than /rmitbot_controller/odom
slamtoolbox	/map /map_metadata /map_updates /slam_toolbox/feedback /slam_toolbox/graph_visualization /slam_toolbox/scan_visualization /slam_toolbox/transition_event /slam_toolbox/update	Loop closure: pose of the odom

5. Future works

- So far, the robot has been spawned inside the unknown virtual environment in Gazebo. The world is just an 8x8m room. It would be appreciated to build a more complicated world to test more reliable scenarios.



Robot Navigation

Table of Contents

[1. Introduction](#)

[2. Tasks](#)

[3. Explanations](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Introduction

The prerequisites are:

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is simulation only. The hardware is not required.

Navigation corresponds to the move from teleoperation robot to autonomous navigation. The robot is given way point to reach. There are heavy tuning to be done to have the robot move in direction of the destination while avoiding obstacles.

2. Tasks

Navigation corresponds to the movement of the robot in the environment, without the user controlling its trajectory. There are infinite paths for the robot to move to the final destination: in a straight line, swirling, going as far as possible from the wall, etc.

Tasks	Code
Install nav2	<code>sudo apt install ros-jazzy-navigation2 ros-jazzy-nav2-bringup -y</code>
Install twist-mux	<code>sudo apt install ros-jazzy-twist-mux</code>
Install twist_stamper	<code>sudo apt install ros-jazzy-twist-stamper</code>
Git clone the repo	<code>git clone https://github.com/ROS2-rmitbot/lesson5_ws.git</code>
Navigate to the workspace, build and source it	<code>cd ~/lesson5_ws rm -rf build install log colcon build source install/setup.bash code .</code>
Launch the bringup launch	<code>ros2 launch rmit_bringup rmitbot.launch.py</code>

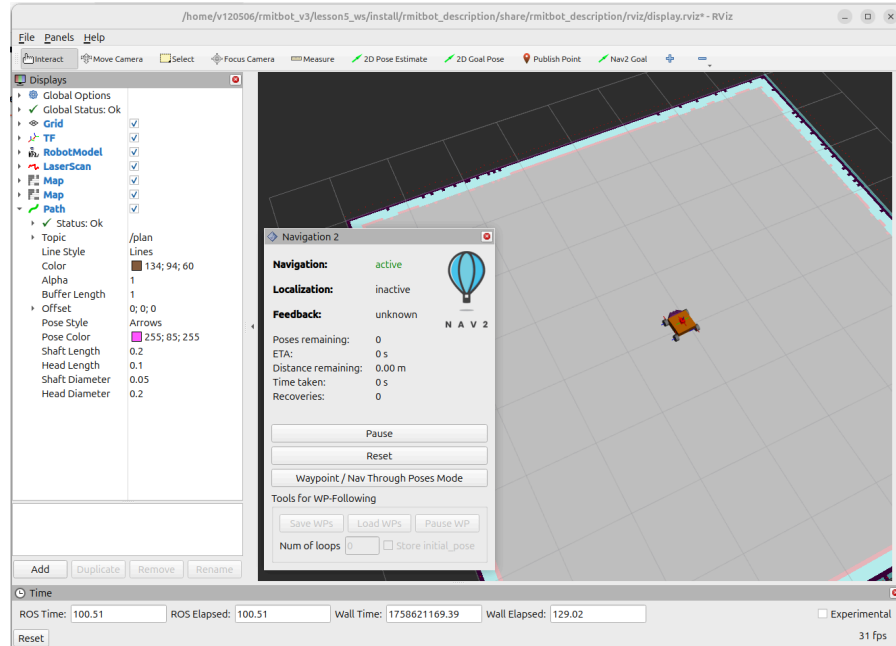


Figure 1: in Rviz, use the goal button to navigate the robot. Add obstacles in Gazebo and see the robot navigate.

3. Explanations

- A world has been added in the URDF.
- The package twistmux has been installed to control the robot with the keyboard (high priority) and with nav2 (low priority).
- The package nav2 has been installed.
- In RViz, select a 2D Goal Pose: the robot should travel to the desired pose autonomously.
- Add some obstacles in Gazebo:
 - o The lidar should detect it.
 - o The robot should navigate and avoid the newly added obstacles.

Note that the robot can be controlled with the teleop keyboard (as usual) and the navigation waypoint in RViz2.

The nav2 package includes several topics. You may check yourself the new topics and nodes launched with the nav2 package:

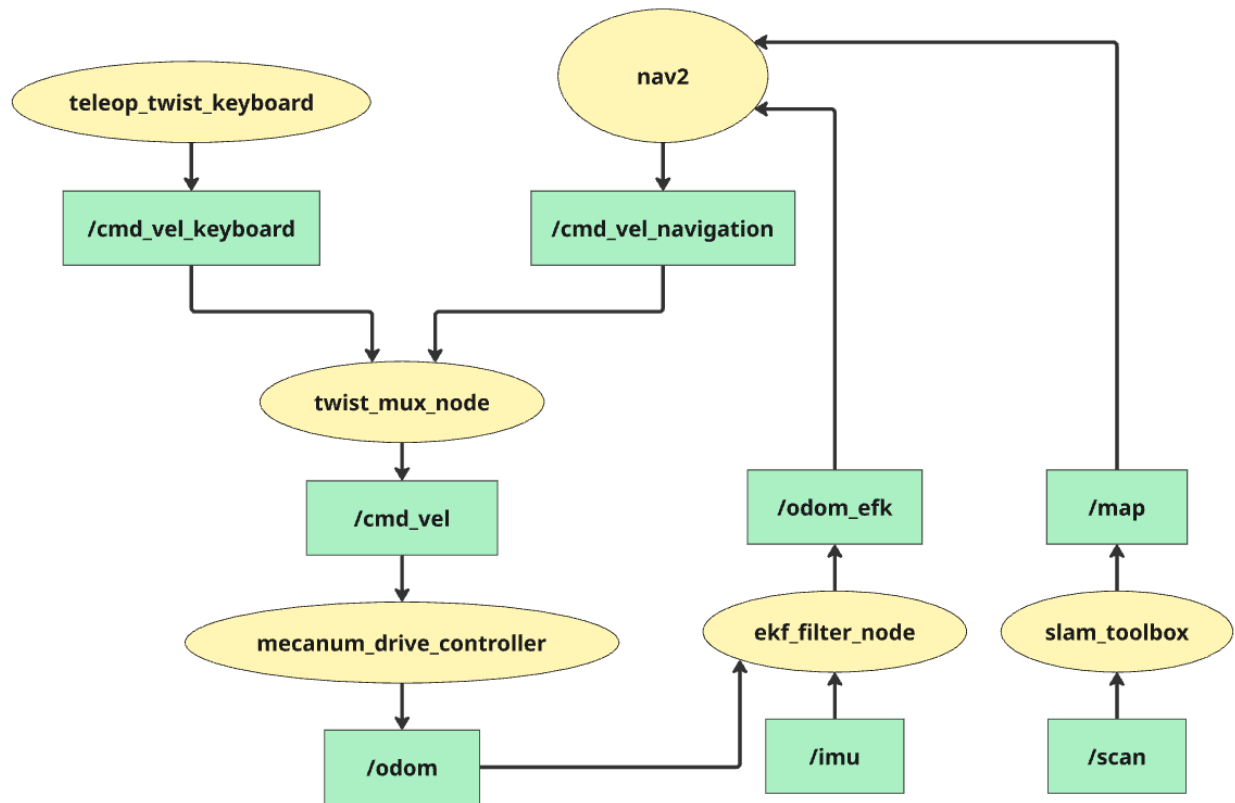
- ros2 node list: list of all the nodes launched
- ros2 topic list: list of all the topics

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup	bringup.launch.py	Launch the other packages
rmitbot_description	rsp.launch.py	robot_state_publisher
	rviz.launch.py	Rviz
	gazebo.launch.py	Gazebo
	jsp_gui.launch.py	Joint_state_publisher_gui (not used anymore)
rmitbot_controller	controller.launch.py	Controller manager: jsb spawner, controller spawner
	teleopkeyboard.launch.py	Launch the input/output of the robot.
rmitbot_localization	local_localization.launch.py	Launch the sensor fusion (ekf) between wheel encoder and imu
rmitbot_mapping	mapping.launch.py	Launch the lidar and slam
rmitbot_navigation	navigation.launch.py	Launch the navigation

5. Future works

- The navigation YAML contains a lot of parameters to tune. It is up to the discretion of the student to fine-tune the robot.
- New topics:
 - /cmd_vel
 - /cmd_vel_keyboard
 - /cmd_vel_navigation
 - /cmd_vel_navigation_unstamped
 - /imu
 - /map
 - /odom
 - /odom_ekf



Robot Vision

Table of Contents

[1. Introduction](#)

[2. Tasks](#)

[3. Explanation](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Introduction

The prerequisites are:

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is simulation only. The hardware is not required.

This package introduces the camera topic and its use with Apriltag detection.

2. Tasks

This package includes a plugin in gazebo for using a camera. In addition, it includes a fiducial marker called apriltag.

Tasks	Code
Install apriltag	<code>sudo apt install ros-jazzy-apriltag-ros</code>
Git clone the repo	git clone https://github.com/ROS2-rmitbot/lesson6_ws.git
Navigate to the workspace, build and source it	<code>cd ~/lesson6_ws</code> <code>rm -rf build install log</code> <code>colcon build</code> <code>source install/setup.bash</code> <code>code .</code>
Launch the bringup launch	<code>ros2 launch rmit_bringup rmitbot.launch.py</code>

Now, you can navigate in the gazebo and you may see an apriltag: this fiducial tag can be detected by the camera.

3. Explanation

Note that:

- The URDF includes a camera.xacro file, which contains a camera plugin for gz.
- The world includes and spawns aprilTag (in addition to the walls).

- The library for the AprilTag can be found at:
<https://github.com/AprilRobotics/apriltag-imgs/tree/master/tag36h11>
- You may need to increase the size. In Ubuntu and in a terminal, you can use:
install imagemagick
sudo apt install imagemagick
convert into a larger picture
convert tag36h11_0.png -resize 600x600 big_tag.png
- In the world, the tag is inserted with a size of 0.162m, which includes the white border. In the apriltag yaml file, the tag should fit only the black border, hence: $6/8 * 0.162 = 0.1215\text{m}$.

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup	bringup.launch.py	Launch the other packages
rmitbot_description	rsp.launch.py rviz.launch.py gazebo.launch.py jsp_gui.launch.py	robot_state_publisher Rviz Gazebo Joint_state_publisher_gui (not used anymore)
rmitbot_controller	controller.launch.py teleopkeyboard.launch.py	Controller manager: jsb spawner, controller spawner Launch the input/output of the robot.
rmitbot_localization	local_localization.launch.py	Launch the sensor fusion (ekf) between wheel encoder and imu
rmitbot_mapping	mapping.launch.py	Launch the lidar and slam
rmitbot_navigation	navigation.launch.py	Launch the navigation
rmitbot_vision	apriltag.launch.py	Include the plugin for camera. Launch the apriltag detection

5. Future works

Thanks to this tag, the robot may know its relative position relative to this fixed and known position (located at [3.8, 0, 0,2]). This tag can be used later on for several purposes, namely:

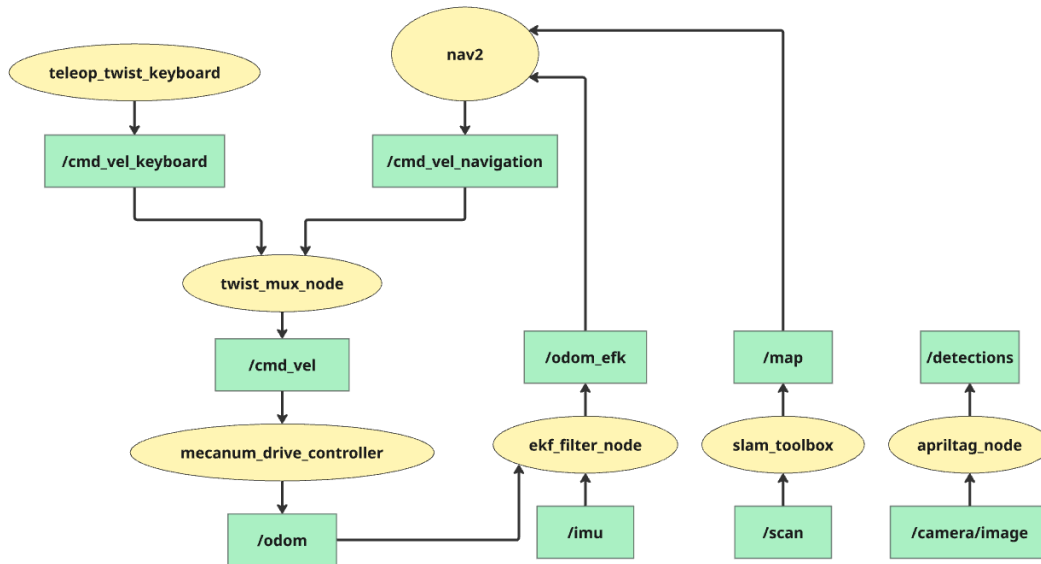
- Accurate positioning of the robot for an indoor environment
- Docking for charging
- If you stick an Apriltag on the robot and use a camera placed on a ceiling, then you can also localize your robot.

The pose estimation and correction can be done with the robot_localization (ekf).

New topics:

- /camera/camera_info

- /camera/image
- /detections



Unit testing: Hardware

Table of Contents

[1. Introduction](#)

[2. DC motor with encoders](#)

[3. IMU ICM20984](#)

1. Introduction

ROS2 is used to control and connect hardware: motor and sensor. However, before connecting the hardware to the ROS2 network, it is essential to verify that they are working well, which is the purpose of the hardware unit testing. The hardware tested in this session is connected to the ESP32, namely:

- DC motors with encoders
- IMU

The hardware connected to the RPI via USB is:

- 2D Lidar
- Webcam

They have an automatic plugin installed by their respective manufacturer. Then there are dedicated ROS2 packages that handle them. Their functioning must be tested, but it requires much less debugging compared to the hardware connected to the ESP32.

2. DC motor with encoders

While testing them, one should verify:

- Does the encoder work?
- Does the motor work?
- Does the PID work?

3. IMU ICM20984

While testing the sensor, one should verify:

- Does the sensor give reasonable data?

DC Motor

Table of Contents

- [1. Introduction](#)
- [2. Microcontroller](#)
- [3. Motor Driver H-Bridge IBT2 \(BTS7960\)](#)
- [4. DC motor with encoder](#)
- [5. Computer](#)
- [6. Power Supply](#)
- [7. Complete setup](#)
 - [7.1. Wiring](#)
 - [7.2. GitHub code cloning](#)
 - [7.3. Code flashing](#)
 - [7.4. Testing](#)

1. Introduction

This document introduces how to drive a DC motor to verify that your hardware is working correctly. The essential components to control the DC motor are given in Figure 1:

- A microcontroller – ESP32
- A motor driver – IBT2
- A DC motor, equipped with encoders – DC12V333RPM
- A computer to code and debug the system
- A power supply – preferably a 12V-10A power supply. Each motor may use up to 3A at startup.

This document gives essential information, but it is up to the student to look for additional knowledge to build a more solid foundation.

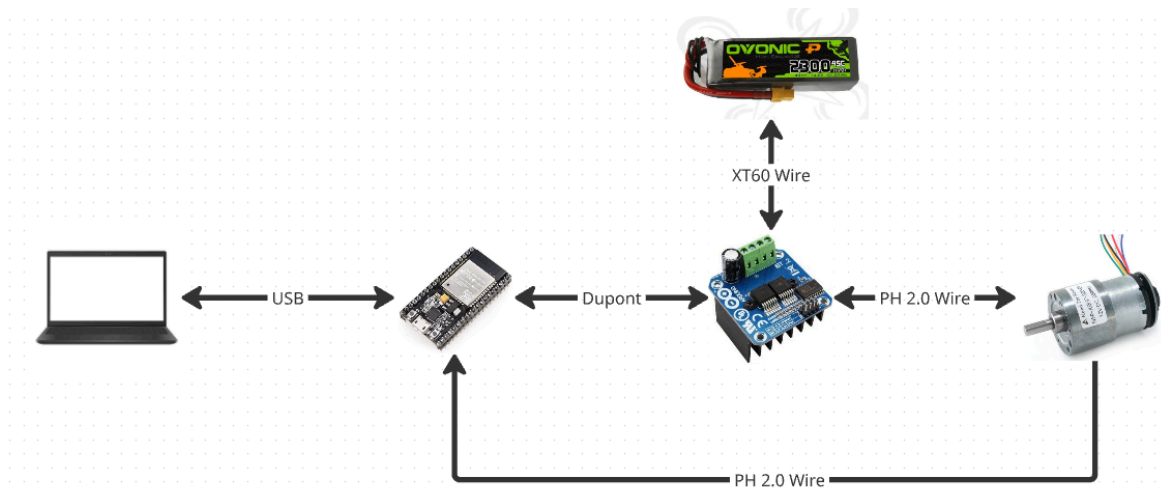


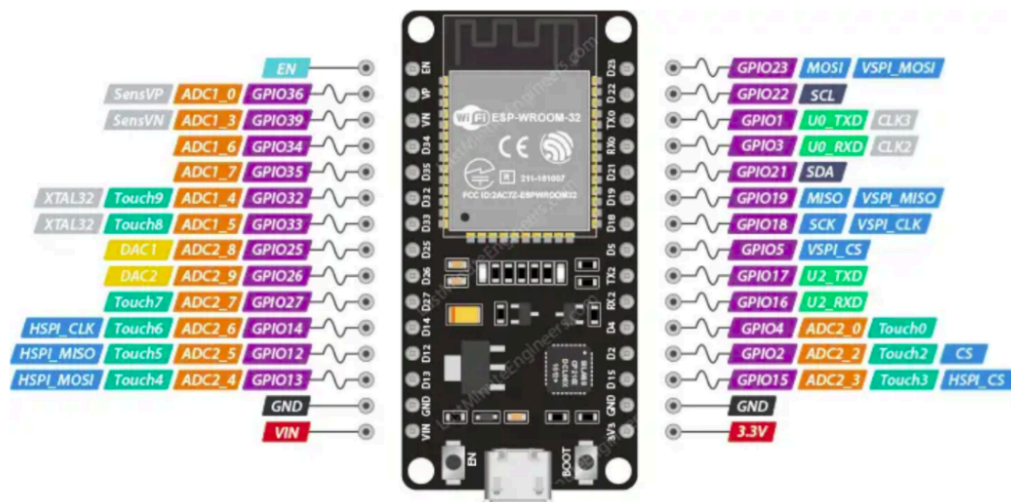
Figure 1: Essential components for driving a DC motor

2. Microcontroller

ESP32 is a microcontroller with Wi-Fi and Bluetooth. It is suitable for the IOT system. In our example, pins 4, 16, 22, 23 are used. Note that these pins are fine to use. Particular care should be considered:

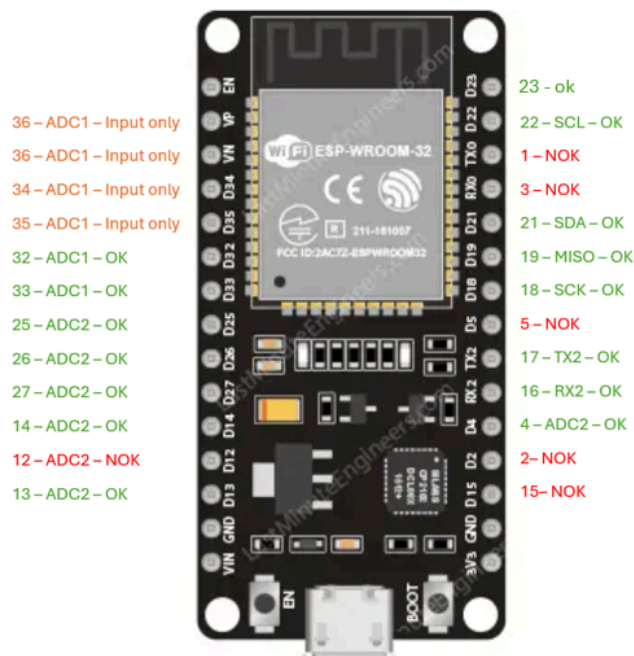
- ADC1 can be used with Wi-Fi, but ADC2 cannot be used with Wi-Fi.
- Some pins cannot be used. Please refer to the following figure.

Other microcontrollers are available: Arduino, STM32, Teensy 4.1. Students are invited to compare existing systems.



Label	GPIO	Safe to use?	Reason				
D0	0	⚠	must be HIGH during boot and LOW for programming	D15	15	⚠	must be HIGH during boot, prevents startup log if pulled LOW
TX0	1	✗	Tx pin, used for flashing and debugging	RX2	16	✓	
D2	2	⚠	must be LOW during boot and also connected to the on-board LED	TX2	17	✓	
RX0	3	✗	Rx pin, used for flashing and debugging	D18	18	✓	
D4	4	✓		D19	19	✓	
D5	5	⚠	must be HIGH during boot	D21	21	✓	
D6	6	✗	Connected to Flash memory	D22	22	✓	
D7	7	✗	Connected to Flash memory	D23	23	✓	
D8	8	✗	Connected to Flash memory	D25	25	✓	
D9	9	✗	Connected to Flash memory	D26	26	✓	
D10	10	✗	Connected to Flash memory	D27	27	✓	
D11	11	✗	Connected to Flash memory	D32	32	✓	
D12	12	⚠	must be LOW during boot	D33	33	✓	
D13	13	✓		D34	34	⚠	Input only GPIO, cannot be configured as output
D14	14	✓		D35	35	⚠	Input only GPIO, cannot be configured as output
D15	15	⚠	must be HIGH during boot, prevents startup log if pulled LOW	VP	36	⚠	Input only GPIO, cannot be configured as output
				VN	39	⚠	Input only GPIO, cannot be configured as output

Figure 2: ESP32 pinout. Note that some pins should not be used.



3. Motor Driver H-Bridge IBT2 (BTS7960)

An H-bridge is an electronic component to drive a DC motor. A power supply could be plugged directly to the DC motor, it will rotate at maximum speed, but can not be modified. Thus, the use of an H-bridge is essential. Refer to the following tutorial to use the H-bridge with a microcontroller: <https://dronebotworkshop.com/dc-motor-drivers/>

In our case, the H-bridge RPWM and LPWM pins are connected to the ESP32 pins 4 and 16. R_EN, L_EN, and 5V should be connected to the 3V3 of the esp32 and the ground of both modules should be connected together.

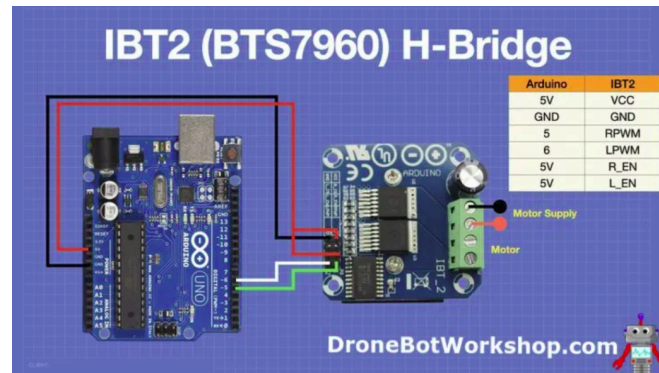


Figure 3: Basic wiring for IBT2 motor driver.

Different H-bridge modules exist: ESC, DRV8833, LN298N. However, these electronic systems may not be suited for the application with the proposed DC motor. Students are expected to justify this statement.

4. DC motor with encoder

The provided DC motor has the following specifications:

- DC motor nominal voltage: 12V
- Gearbox: 1:30
- Encoder resolution: 11 pulses per revolution

For one rotation of the output shaft, the encoder counts 330 pulses. Building a mechatronics system requires motors and sensors. The students are invited to compare different existing technologies: BLDC, stepper, AC motors, etc.

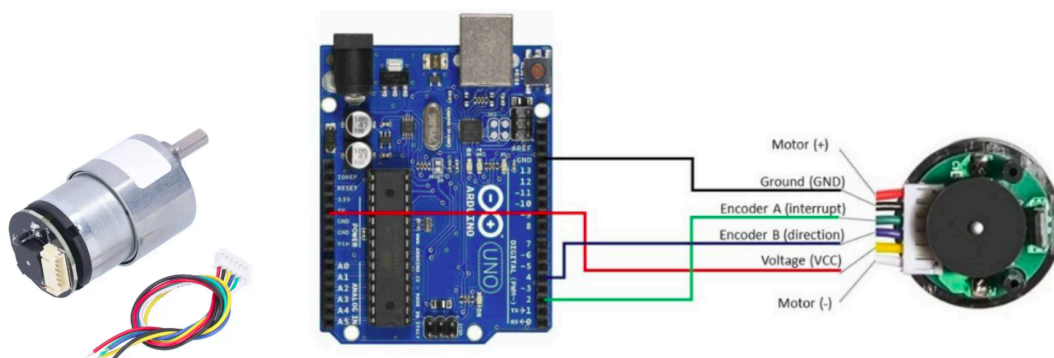


Figure 4: Basic wiring for DC motor with encoders.

A quadrature encoder gives the angular position of the motor. The functioning of the quadrature encoder should be explained by the student. In our example, the encoder A and B are connected to the esp32 pins 22 and 23.

5. Computer

The computer (PC) is connected to the ESP32 via USB. The communication between them is based on serial communication. In terms of coding environment, Arduino IDE can be used, but can be quite limited with additional libraries (encoder, PID, maybe later IMU libraries, etc.). A preferred coding interface is **VSCode**, which is used in software engineering for web development and embedded systems.

VSCode website: <https://code.visualstudio.com/>

However, VSCode is quite a general IDE for coding in html, css, js, cpp, etc. The preferred support for embedded systems is the add-in **platformIO**. One advantage of this tool is the ability to integrate different libraries developed by the community.

PlatformIO website: <https://platformio.org/>

The best would be to follow a tutorial to install VSCode, add platformIO, and get started with simple code: <https://youtu.be/JmvMvlphMnY?si=IcGgeldb5zOA86KR>

6. Power Supply

In terms of power supply, the DC motor is rated with a voltage of 12V, and a current of 1A. However, at startup, the motor can use up to 3A to overcome the static friction. A fixed power supply, e.g. a power adapter, can be used at first to test the system. Later on a battery would be preferred for mobile robots. In this case, several options are available: LIPO, Li-Ion, LifePO4. It is up to the student to do their diligent literature to find the most suitable option.

7. Complete setup

Please follow the previous steps before following this last part about the complete setup.

7.1. Wiring

You may use a breadboard for unit testing purposes. However, the final robot consists of four motors and the use of a breadboard is highly undesirable. The use of a screw terminal could be a better option, but a custom PCB would be the most appropriate solution.

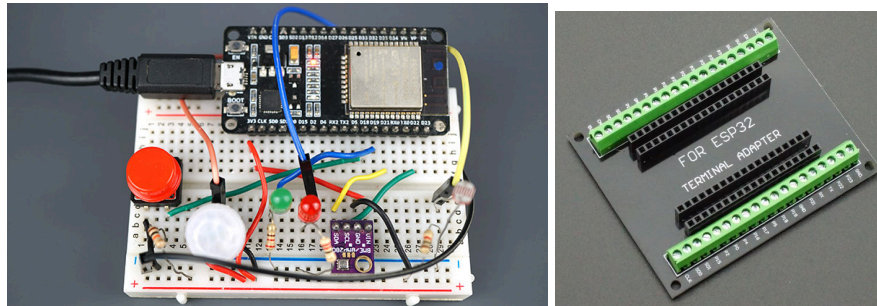


Figure 5: Breadboard will likely have loose connection. Prefer a terminal screw for ESP32

The pins used on the esp32 for the encoders are 22, 23. The pins used on the esp32 for the motor driver are 4, 16. You can refer to the file MySetup.h in the include folder while the code will be downloaded from GitHub.

7.2. GitHub code cloning

The code is available on GitHub. You may download the zip file containing the file, but a more appropriate approach is to learn how to clone the code using a terminal on your computer. This would be helpful especially while working with a terminal through ssh. The code is available at: <https://github.com/ROS2-AutoBot/04-DCMotorControl>

In the terminal, use “cd” to change directory and “ls” to list the items in the current folder. Then use the next function to create a folder that contains the code:

git clone <https://github.com/ROS2-AutoBot/04-DCMotorControl>

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\v120506> git clone https://github.com/ROS2-AutoBot/04-DCMotorControl
```

Figure 6: A PC terminal uses the command line to execute a function.

7.3. Code flashing

Flashing the code to the ESP32 is straightforward with VSCode and PlatformIO. Make sure the pin for the PWM pin for the motor driver and the encoder pin from the MySetup.h file match the wiring. With PlatformIO, select the folder that has been created “04-DCMotorControl”. Make sure that this folder contains the file platformio.ini. Then, upload the code to the esp32.

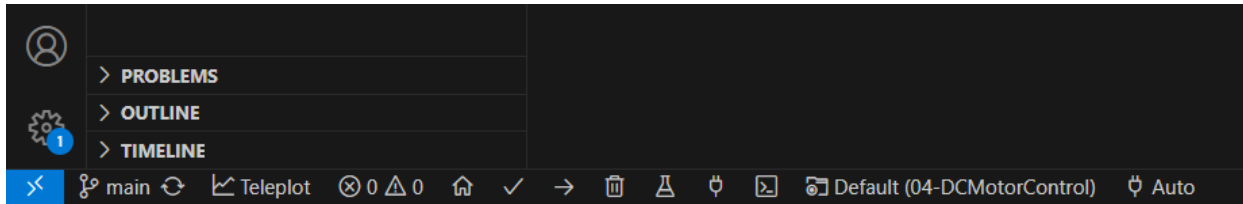


Figure 6: The interface with PlatformIO may be overwhelming. At the bottom of the screen, you may see the basic function required for compiling and flashing your code.

7.4. Testing

The file MySerial.cpp contains the code to communicate via serial with the esp32. The serial communication allows the next tasks:

- Esp32 is sending the information to the computer: speed measurement, speed reference, motor command, and encoder tick.
- The computer is sending the desired speed reference to the esp32.

Open the serial communication and type “q10”: normally you should see the DC motor rotating at 10 rad/s. You may try other values as well. An important part of the unit testing is to verify that the actual speed of the motor is correct: you may ask for a reference speed of 10 rad/s. The encoder shows that the speed is 10 rad/s, but the motor may rotate at a totally incorrect speed. The use of a tachometer as an external source of measurement would be preferred, but you may set a relatively reasonable speed to manually calculate the speed of the motor. For instance, you may set a speed of 6.28 rad/s (about 1 rotation per second) and count the actual rotation of the motor shaft per second. This would give a first approximation about the correctness of the motor speed.

An interesting axis of analysis would be control systems. You may want to tune the PID controller depending on your desired dynamics. It is possible to visualize the data with the tool

called “teleplot” in VSCode. With the data collection regarding the motor speed and the motor command, it would be possible to:

- Perform model identification
- Use Simulink to select the PID gains to respect overshoot, rise time, settling time, and steady-state error specification.
- Use these found PID gains with the real system and compare the behaviour between the modelled system and the real physical one.

IMU: ICM20948

Table of Contents

[1. Introduction](#)

[2. Microcontroller](#)

[3. IMU ICM20948](#)

1. Introduction

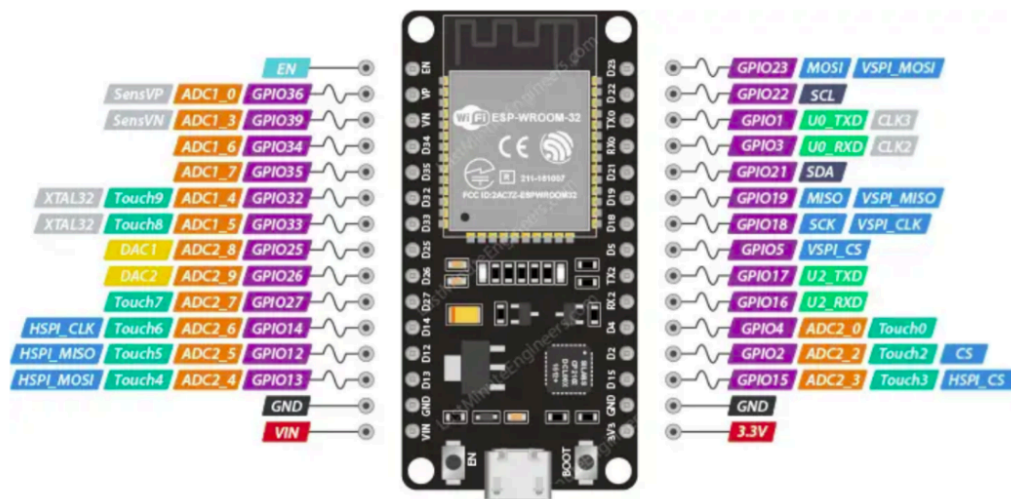
The inertial measurement unit (IMU) is essential for robot odometry: the wheel encoder is not reliable enough, especially because the robot will slip. In this case, the IMU ICM20948 will provide the orientation of the robot. The sensor fusion is performed via the package robot_localization. However, the IMU must be tested and verified before its integration into the robot system.

2. Microcontroller

ESP32 is a microcontroller with Wi-Fi and Bluetooth. It is suitable for the IOT system. In our example, pins 4, 16, 22, and 23 are used. Note that these pins are fine to use. Particular care should be considered:

- ADC1 can be used with Wi-Fi, but ADC2 cannot be used with Wi-Fi.
- Some pins cannot be used. Please refer to the following figure.

Other microcontrollers are available: Arduino, STM32, Teensy 4.1. Students are invited to compare existing systems.



Label	GPIO	Safe to use?	Reason				
D0	0	⚠	must be HIGH during boot and LOW for programming		D15	15	⚠ must be HIGH during boot, prevents startup log if pulled LOW
TX0	1	✗	Tx pin, used for flashing and debugging		RX2	16	✓
D2	2	⚠	must be LOW during boot and also connected to the on-board LED		TX2	17	✓
RX0	3	✗	Rx pin, used for flashing and debugging		D18	18	✓
D4	4	✓			D19	19	✓
D5	5	⚠	must be HIGH during boot		D21	21	✓
D6	6	✗	Connected to Flash memory		D22	22	✓
D7	7	✗	Connected to Flash memory		D23	23	✓
D8	8	✗	Connected to Flash memory		D25	25	✓
D9	9	✗	Connected to Flash memory		D26	26	✓
D10	10	✗	Connected to Flash memory		D27	27	✓
D11	11	✗	Connected to Flash memory		D32	32	✓
D12	12	⚠	must be LOW during boot		D33	33	✓
D13	13	✓			D34	34	⚠ Input only GPIO, cannot be configured as output
D14	14	✓			D35	35	⚠ Input only GPIO, cannot be configured as output
D15	15	⚠	must be HIGH during boot, prevents startup log if pulled LOW		VP	36	⚠ Input only GPIO, cannot be configured as output
					VN	39	⚠ Input only GPIO, cannot be configured as output

36 – ADC1 – Input only
 36 – ADC1 – Input only
 34 – ADC1 – Input only
 35 – ADC1 – Input only
 32 – ADC1 – OK
 33 – ADC1 – OK
 25 – ADC2 – OK
 26 – ADC2 – OK
 27 – ADC2 – OK
 14 – ADC2 – OK
 12 – ADC2 – NOK
 13 – ADC2 – OK

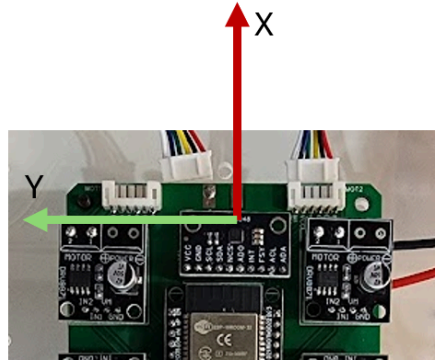
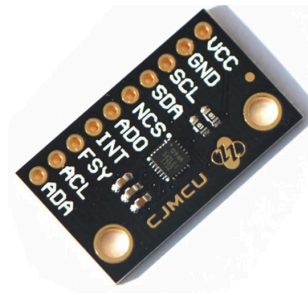


23 - ok
 22 – SCL – OK
 1 – NOK
 3 – NOK
 21 – SDA – OK
 19 – MISO – OK
 18 – SCK – OK
 5 – NOK
 17 – TX2 – OK
 16 – RX2 – OK
 4 – ADC2 – OK
 2 – NOK
 15 – NOK

3. IMU ICM20948

The IMU provides the following information:

- Angular position via quaternion
- Angular velocity in rad/s
- Linear acceleration in m/s².



Be careful about the direction of the sensor: it would be easier to follow the ROS2 convention (x-axis forward, y-axis to the left, z-axis up). The ICM20948 uses an integrated chip called the DMP: it outputs the required data directly, but it is sensitive to noise.

The code for the ESP32 is located in the following repository:

https://github.com/ROS2-rmitbot/lesson8_ws/tree/main/src/rmitbot_firmware/arduino_firmware/rmitbot_arduino_firmware

Note: you may need to modify one of the files from the library:

- Find SparkFun_ICM-20948_ArduinoLibrary/src/util/ICM_20948_C.h
- Uncomment `#define ICM_20948_USE_DMP` (should be on line 34)

Verify the following data:

- Easiest: check the gravitational acceleration: is it in the correct direction?
- Normal: check the gyro data: when you rotate the module in the x-axis, does it get positive/negative?
- Hardest: from the quaternion, can you verify the angular position in degree?

ROS2 with Hardware

Table of Contents

[1. Introduction](#)

1. Introduction

Similar to the unit testing of the hardware connected to the ESP32, the hardware here will be tested for each package.

- Description: There is no hardware involved
- Controller: DC motor
- Localization: IMU
- Mapping Lidar
- Navigation: tuning of the navigation parameters for the real robot
- Vision: web camera

Robot Controller - ESP32

Table of Contents

[1. Introduction](#)

[2. Tasks](#)

[2.1. With the ESP32 connected to the computer](#)

[2.2. With the ESP32 connected to the RPI](#)

[2.3 Clock between PC and RPI](#)

[3. Explanation](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Introduction

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is NOT a simulation. The gazebo is no longer used. The hardware is required.
- VSCode and its extension PlatformIO (PIO) should be installed.
- First, connect the ESP32 to the computer to test the communication between ROS2 and the hardware. Later, it would be mandatory to connect the RPI to the ESP32.

The ESP32 and the motors have been verified and should function. Now it is a good time to test if the communication between ROS2 and the ESP32 functions for the motors. Unit testing is important because if at this point the motor does not function:

- Is it because of the motor itself?
- Is it because of the ESP32?
- Is it because of the communication with the RPI?

2. Tasks

It is now time to use the real hardware. So far, Gazebo has been used to get comfortable with the ROS2 environment and packages. Gazebo was in charge of broadcasting the `joint_state`. Now the hardware package, included in this lesson, listens to the serial communication (`esp32`) and publishes the `joint_state`. Note that the ESP32 firmware must be flashed on your ESP32 microcontroller. It can be found in the hardware package in the following folder:

`src/rmitbot_firmware/arduino_firmware/rmitbot_arduino_firmware`

Please follow the guidelines relative to VSCode, PIO for flashing software:

<https://randomnerdtutorials.com/vs-code-platformio-ide-esp32-esp8266-arduino/>

2.1. With the ESP32 connected to the computer

Tasks	Code
Install the serial communication library	<code>sudo apt install libserial-dev</code>
Clone the repo from GitHub	<code>git clone https://github.com/ROS2-rmitbot/lesson7_ws.git</code>
Navigate to the workspace	<code>cd ~/lesson7_ws</code>
Remove folders: build, install, log	<code>rm -rf build install log</code>
Build the workspace	<code>colcon build</code>
Source the workspace	<code>source install/setup.bash</code>
Open VSCode	<code>code .</code>
Launch the bringup launch	<code>ros2 launch rmitbot_bringup rmitbot.launch.py</code>

Use the teleop keyboard to move the robot around and verify that the real robot and the robot in RViz are moving similarly. You may add and visualize the imu topic in RViz as well. Its integration with robot_localization (ekf) is performed in the next lesson.

2.2. With the ESP32 connected to the RPI

So far, the computer is connected to the ESP32. A more common setup is as shown in Figure 1.

Usually, to SSH into your RPI, you may open a terminal and enter the command:

`ssh username@ipaddr.`

However, one may use the VSCode SSH capabilities. Please look at the following link:

<https://youtu.be/jvi1nmKK81Y?si=ASWLSq4Oun-zVKg9>

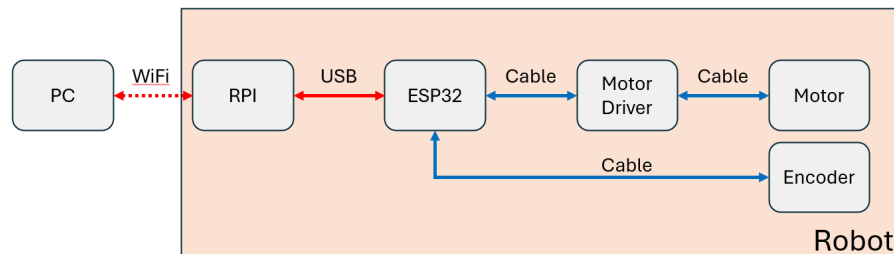


Figure 1: Robot configuration and connection

Tasks	Code
Start SSH (Make sure the RPI and PC are connected to the same WiFi)	Open terminal → <code>ssh username@ipaddr</code>
On the RPI	
Install previous libraries: ros2_control	<code>sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers</code>
Install previous libraries: xacro	<code>sudo apt install ros-jazzy-xacro</code>

Install the serial communication library	sudo apt install libserial-dev
On the RPI Clone the repo from GitHub	git clone https://github.com/ROS2-rmitbot/lesson7_ws.git
On the RPI Navigate to the workspace, build, and source it	cd ~/lesson7_ws rm -rf build install log colcon build source install/setup.bash
On the RPI Launch the rsp Launch the controller	ros2 launch rmit_description rsp.launch.py ros2 launch rmit_controller controller.launch.py
On the PC Open a new terminal Start rviz2	ros2 launch rmitbot_description display.launch.py
On the PC Start teleopkeyboard	ros2 launch rmitbot_controller teleopkeyboard.launch.py

You should be able to control the real robot from the teleoperation keyboard of the PC.

2.3 Clock between PC and RPI

You may experience some delay between the RPI and the PC. On Wi-Fi, the ping should be around 10ms. However, if the clock is not synchronised between the two devices, a more significant lag may be experienced.

Tasks	Code
Check if the ping is fine with the RPi	ping ipaddr
Install chrony on each device	sudo apt install chrony -y
Check status	timedatectl Normally, you should have: System clock synchronized: yes NTP service: active

3. Explanation

Few modifications have been performed in the packages:

- The use_sim_time parameter has been set to false in all packages
- The URDF is calling the hardware interface instead of the Gazebo plugin.
- For the moment, only the hardware connected to the ESP32 is tested. The integration of the Lidar and camera is covered in the next packages.

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup		Launch the other packages and nodes
rmitbot_description	display.launch.py: rviz, rsp gazebo.launch.py: gazebo	Launch the URDF and static tf
rmitbot_controller	Controller manager: jsb spawner, controller spawner	Launch the input/output of the robot
rmitbot_firmware	Arduino firmware Hardware interface	The Arduino firmware is the code flashed on the ESP32. The hardware interface is the node launched with ROS2.

5. Future works

Verify that your real hardware robot is moving in the same fashion as your robot in RViz. If there is a discrepancy, could you measure it?

Robot Localization - ICM20948

Table of Contents

[1. Introduction](#)

[2. Tasks](#)

[2.1. With the esp32 connected to the computer](#)

[2.2. With the ESP32 connected to the RPI](#)

[3. Explanation](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Introduction

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is NOT a simulation. Gazebo is no longer used. The hardware is required.
- VSCode and its extension PlatformIO (PIO) should be installed.
- First, connect the ESP32 to the computer to test the communication between ROS2 and the hardware. Later, it would be mandatory to connect the RPI to the ESP32.

The code flashed on the ESP32 should work in this lesson as well. The `robot_localization` package is introduced with the use of the IMU.

2. Tasks

Make sure that the hardware interface is working. Normally, the joint state and the imu should already be available from the hardware interface. This lesson focuses on the integration of the hardware imu with the package `robot_localization`

2.1. With the ESP32 connected to the computer

Tasks	Code
Clone the repo from github	git clone https://github.com/ROS2-rmitbot/lesson8_ws.git
Navigate to the workspace, build and source it	<pre>cd ~/lesson8_ws rm -rf build install log colcon build source install/setup.bash code .</pre>
Launch the bringup launch	<pre>ros2 launch rmitbot_bringup rmitbot.launch.py</pre>

Carry your robot and rotate it in the z-axis (yaw). The robot should rotate in rviz. The red arrow corresponds to the odometry from the wheel. The green arrow corresponds to the fused odometry from the robot_localization (imu+encoder).

Note: you may need to modify one of the file from the library:

- Find SparkFun_ICM-20948_ArduinoLibrary/src/util/ICM_20948_C.h
- Uncomment #define ICM_20948_USE_DMP (should be lined 34)

2.2. With the ESP32 connected to the RPI

This should be straightforward if the previous lesson has been performed. The only difference is the use of the imu with the robot_localization package.

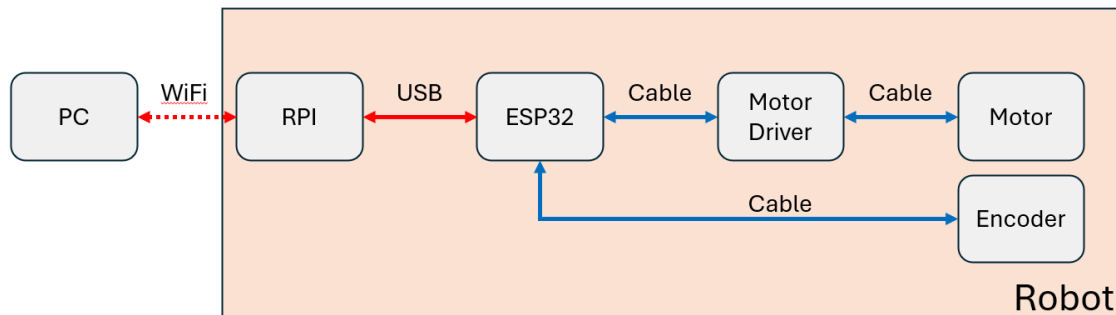


Figure 1: Robot configuration and connection

Tasks	Code
Start SSH (make sure the RPI and PC are connected on the same WiFi)	Open terminal → ssh username@ipaddr
On the RPI Install serial communication library	sudo apt install libserial-dev
On the RPI Clone the repo from github	git clone https://github.com/ROS2-rmitbot/lesson8_ws.git
On the RPI Navigate to the workspace, build and source it	cd ~/lesson8_ws rm -rf build install log colcon build source install/setup.bash
On the RPI Launch the controller	ros2 launch rmit_controller controller.launch.py
On the PC Open a new terminal Start rviz2	ros2 launch rmitbot_description display.launch.py

On the PC

Start teleopkeyboard

```
ros2 launch rmitbot_controller teleopkeyboard.launch.py
```

You should be able to control the real robot from the teleoperation keyboard of the PC.

3. Explanation

There are few modifications that have been performed in the packages:

- use_sim_time parameter has been set to false in all packages
- The urdf is calling the hardware interface instead of the gazebo plugin.
- For the moment, only the hardware connected to the esp32 is tested. The integration of the Lidar and camera is covered in the next packages.

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup		Launch the other packages and nodes
rmitbot_description	display.launch.py: rviz, rsp gazebo.launch.py: gazebo	Launch the urdf and static tf
rmitbot_controller	Controller manager: jsb spawner, controller spawner	Launch input/output of the robot
rmitbot_localization	ekf	Launch the sensor fusion between wheel encoder and imu
rmitbot_mapping	slam	Launch the lidar and slam
rmitbot_navigation	nav2	Launch the navigation
rmitbot_vision	apriltag	Include the plugin for camera. Launch the apriltag detection
rmitbot_firmware	Arduino firmware Hardware interface	The arduino firmware is the code flashed on the esp32. The hardware interface is the node launched with ROS2.

5. Future works

Verify that your real hardware robot is moving in the same fashion as your robot in rviz. If there is discrepancy, could you measure it?

Robot Mapping - RPLidar A1M8

Table of Contents

[1. Pre-requisites](#)

[2. Tasks](#)

[2.1. With the Rplidar A1M8 connected on the computer](#)

[2.2. With the Rplidar A1M8 connected on the RPI](#)

[2.3. Consideration regarding serial port name with Ubuntu](#)

[3. Explanation](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Pre-requisites

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is NOT a simulation. Gazebo is no longer used. The hardware is required.
- VSCode and its extension PlatformIO (PIO) should be installed.
- First, connect the ESP32 to the computer to test the communication between ROS2 and the hardware. Later, it would be mandatory to connect the RPI to the ESP32.

2. Tasks

It is essential to verify that the lidar works, first with the device connected to the PC, then connected to the RPI. Documentation about the rplidar: https://index.ros.org/p/rplidar_ros/#jazzy

2.1. With the Rplidar A1M8 connected on the computer

At first it is necessary to verify that the Rplidar A1M8 is working. Please install the required packages.

Tasks	Code
Plug the RPLidar to the computer	
Install the package for the lidar hardware (rplidar A1M8)	<code>sudo apt install ros-jazzy-rplidar-ros</code>
Run the lidar (hardware) Be aware about the name of the serial port: <code>/dev/ttyUSB0</code>	<code>ros2 run rplidar_ros rplidar_composition --ros-args -p serial_port:=/dev/ttyUSB0 -p frame_id:=lidar_link -p angle_compensate:=true -p scan_mode:=Standard</code>
Or use the launch file	<code>ros2 launch rmitbot_mapping rplidar.launch.py</code>
Run rviz2	<code>ros2 run rviz2 rviz2</code>

You should be able to visualize the cloud point from the lidar in rviz.

2.2. With the Rplidar A1M8 connected on the RPI

So far, the computer is connected to the Lidar. A more common setup is as shown in Figure 1.

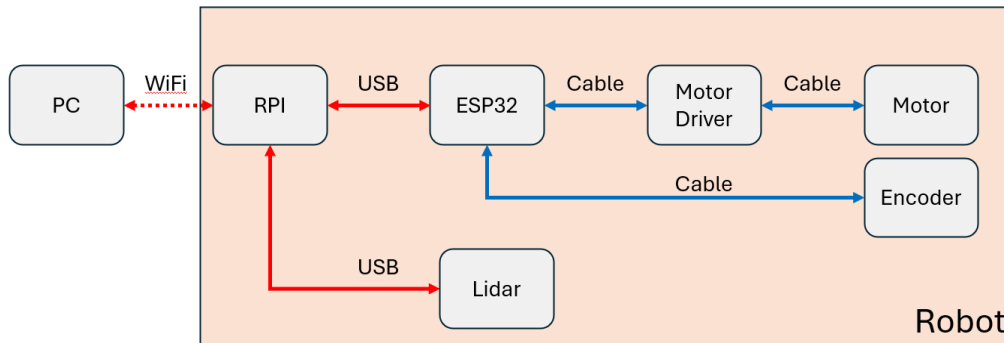


Figure 1: Robot configuration and connection

Tasks	Code
Start SSH (make sure the RPI and PC are connected on the same WiFi)	<code>ssh alpha-pi@alpha-pi.local</code>
On the RPI Install the package for the lidar hardware (rplidar A1M8)	<code>sudo apt install ros-jazzy-rplidar-ros</code>
On the RPI Run the lidar (hardware)	<code>ros2 run rplidar_ros rplidar_composition --ros-args -p serial_port:=/dev/ttyUSB0 -p frame_id:=lidar_link -p angle_compensate:=true -p scan_mode:=Standard</code>
Or use the launch file	<code>ros2 launch rmitbot_mapping rplidar.launch.py</code>
On the PC Run rviz2	<code>ros2 run rviz2 rviz2</code>

You should be able to run the lidar from the RPI via SSH from the computer.

2.3. Consideration regarding serial port name with Ubuntu

Ubuntu dynamically changes the USB port name: if you connect your esp32 and your lidar, the usb name might change. Thus, it is advised to develop a user development rule (udev), on your RPI and on your PC, so that the name does not change when you plug your peripheral.

Tasks	Code
Use the command <code>lsusb</code> to know the information of your serial device	<code>lsusb</code> E.g.: esp32 usb information Bus 003 Device 020: ID 1a86:7523 QinHeng Electronics CH340 serial converter

Launch nano (text editing software in terminal):	<code>sudo nano /etc/udev/rules.d/99-serial-aliases.rules</code>
Copy the following lines:	<code>SUBSYSTEM=="tty",ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", SYMLINK+="esp32"</code> <code>SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", SYMLINK+="rplidar"</code>
Save:	Ctrl + O, Press Enter
Close:	Ctrl + X
in a terminal, launch the next CLI:	<code>sudo udevadm control --reload-rules</code> <code>sudo udevadm trigger</code>
Verify by launching: You should see:	<code>ls /dev/*</code> <code>/dev/esp32</code> <code>/dev/rplidar</code>

3. Explanation

There are few modifications that have been performed in the packages:

- `use_sim_time` parameter has been set to false in all packages
- The urdf is calling the hardware interface instead of the gazebo plugin.
- The integration of the Lidar hardware is located in `rmitbot_mapping`.

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup		Launch the other packages and nodes
rmitbot_description	<code>display.launch.py</code> : rviz, rsp <code>gazebo.launch.py</code> : gazebo	Launch the urdf and static tf
rmitbot_controller	Controller manager: jsb spawner, controller spawner	Launch input/output of the robot
rmitbot_localization	ekf	Launch the sensor fusion between wheel encoder and imu
rmitbot_mapping	Slam rplidar	Launch the lidar and slam
rmitbot_navigation	nav2	Launch the navigation
rmitbot_vision	apriltag	Include the plugin for camera. Launch the apriltag detection
rmitbot_firmware	Arduino firmware Hardware interface	The arduino firmware is the code flashed on the esp32. The hardware interface is the node launched with ROS2.

5. Future works

The Rplidar should be launched from your RPI. The slamtoolbox should be launched from your computer. Can you move your robot and start to do the mapping of the area around the robot?

Robot Navigation - Tuning the gains

Table of Contents

[1. Pre-requisites](#)

[2. Tasks](#)

[3. Explanation](#)

[4. Packages, nodes, and topics](#)

[5. Future works](#)

1. Pre-requisites

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is NOT a simulation. Gazebo is no longer used. The hardware is required.
- VSCode and its extension PlatformIO (PIO) should be installed.
- The esp32 and the lidar should have been tested while being connected on the RPI. For this package, the robot should NOT be connected to the computer.

2. Tasks

There are no packages to be installed, but make sure that the PC and RPI launch the right file respectively:

- PC should launch: rviz, slam, nav2
- RPI should launch: controller, rplidar

The bringup file for each platform should be modified to launch the correct packages.

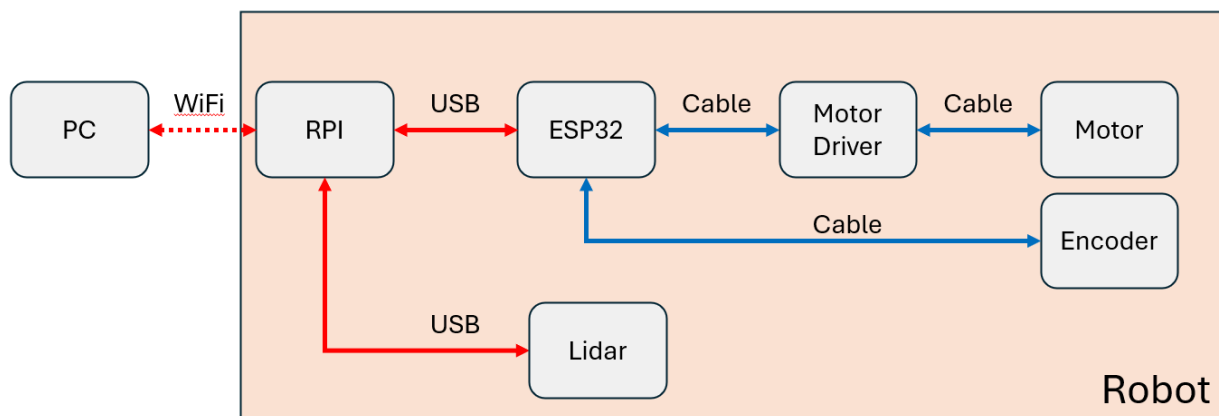


Figure 1: Robot configuration and connection

3. Explanation

One of the benefits of ROS2 is its abstraction regarding communication: as long as the RPI and PC are on the same network, they share the topics and nodes.

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup		Launch the other packages and nodes
rmitbot_description	display.launch.py: rviz, rsp gazebo.launch.py: gazebo	Launch the urdf and static tf
rmitbot_controller	Controller manager: jsb spawner, controller spawner	Launch input/output of the robot
rmitbot_localization	ekf	Launch the sensor fusion between wheel encoder and imu
rmitbot_mapping	Slam rplidar	Launch the lidar and slam
rmitbot_navigation	nav2	Launch the navigation
rmitbot_vision	apriltag	include the plugin for camera. Launch the apriltag detection
rmitbot_firmware	Arduino firmware Hardware interface	The arduino firmware is the code flashed on the esp32. The hardware interface is the node launched with ROS2.

5. Future works

The robot may require fine tuning, as the simulation and the real hardware are rarely perfectly matching.

Robot Vision - Webcam

Table of Contents

[1. Pre-requisites](#)

[2. Installing the camera on the RPI](#)

[3. Testing the camera](#)

[5. Future works](#)

1. Pre-requisites

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is NOT a simulation. Gazebo is no longer used. The hardware is required.
- VSCode and its extension PlatformIO (PIO) should be installed.
- This package only introduces how to get started with the camera module 3.

2. Installing the camera on the RPI

The camera module v3 for RPI requires special attention for installation of the packages. If a USB camera is used, the process is much simpler. Using the camera module with the RPI is quite fragile. The latch to install the camera module can be tricky. Have a look at the following videos:

<https://youtu.be/IAbpDRy-gc0?si=pMC79ZBYrfffU3NP>

<https://youtu.be/6poDg63DLVM?si=0s8pCWwj3Q08mxrH>

Reference for installing the camera package:

https://github.com/christianrauch/camera_ros

Tasks	Code
Remote control the rpi via ssh	ssh rpi@ipaddr
Install dependencies	sudo apt update sudo apt install -y git build-essential meson ninja-build pkg-config libyaml-dev python3-pip sudo apt install -y libboost-dev libgnutls28-dev openssl libssl-dev libevent-dev sudo apt install -y qtbase5-dev libqt5core5a libqt5gui5 libqt5widgets5 sudo apt install python3-ply
Create workspace	mkdir -p ~/camera_ws/src cd ~/camera_ws/src
Install python3-colcon-meson	sudo apt -y install python3-colcon-meson
Get libcamera for camera module	git clone https://github.com/raspberrypi/libcamera.git

Get camera_ros	git clone https://github.com/christianrauch/camera_ros.git
Resolve binary dependencies and build workspace	source /opt/ros/jazzy/setup.bash cd ~/camera_ws/
Finally, reboot the pi	sudo rosdep init rosdep update rosdep install -y --from-paths src --ignore-src --rosdistro jazzy --skip-keys=libcamera colcon build --event-handlers=console_direct+ sudo reboot

3. Testing the camera

Tasks	Code
Remote control the RPI via ssh	ssh rpi@ipaddr
On the RPI Launch the camera node	cd ~/camera_ws/ source /opt/ros/jazzy/setup.bash source install/setup.bash ros2 run camera_ros camera_node
On the PC Launch rviz	ros2 run rviz2 rviz2 add ▢ image ▢ camera/image_raw

Normally, in rviz, there is a topic called image, which can be selected to watch the stream from the camera. By default, the stream uses the highest resolution, which is likely yielding lagging video. To correct this, run the camera with lower resolution and listen to the *compressed topic.

Tasks	Code
Remote control the rpi via ssh	ssh rpi@ipaddr
On the RPI Launch the camera node	cd ~/camera_ws/ source install/setup.bash ros2 run camera_ros camera_node --ros-args -p width:=320 -p height:=240 -p format:=YUYV
On the PC Launch rviz	ros2 run rviz2 rviz2 add ▢ image ▢ camera/image_raw/compressed

5. Future works

The robot is now complete. The camera feed is useful to navigate with the robot, however, much more intelligent features could be added with this camera, such as:

- Better localization with apriltag

- Computer vision: image recognition, object following, etc.

ROS2 Advanced Concepts

Table of Contents

- [1. Introduction](#)
- [2. Controlling the robot over the internet - Tailscale](#)
- [3. Dealing with latency - Zenoh](#)
- [4. Setup a joystick](#)
- [5. Advanced Navigation - AprilTag](#)
- [6. Object recognition - OpenCV](#)
- [7. 3D mapping - OAK-D Lite camera](#)
- [8. Visualization for Engineers - Foxglove](#)
- [9. Visualization for clients - Cloud Computing and Tunneling](#)
- [10. Multirobot control - namespace and open-RMF](#)

1. Introduction

The previous chapters are the fundamentals for getting started with autonomous navigation. The following chapters give only the general information regarding the upgrade of the current robots.

2. Controlling the robot over the internet - Tailscale

So far, the robot has been connected and controlled over the local WiFi. How can we set up the robot to be controlled over the internet?

- The RPI can connect to the internet (e.g., the University WiFi)
- Your computer can be connected from your house
- How do they find each other? They should be on the same private network
- A VPN is a virtual private network that creates a local network over the internet. Tailscale is the easiest tool to use.

3. Dealing with latency - Zenoh

When data becomes quite large and the bandwidth is limited, the RPI will disconnect from the WiFi, or the latency will be noticeable, even with high-speed internet. This is due to the DDS communication from ROS2, which is useful for Ethernet communication, but not for WiFi. Zenoh is the new communication protocol, which solves the problem of having scarce WiFi (and the Zenoh communication protocol does much more!).

4. Setup a joystick

A Bluetooth joystick is recommended, as it is more flexible. Using a VM, Bluetooth can be activated in Ubuntu or Windows, but not both. A joystick is a good addition to control the robot manually, as the teleop keyboard can be less intuitive to use. The joystick must be integrated with twistmux, as the teleopkeyboard and the navigation stack are publishing to `/cmd_vel`.

5. Advanced Navigation - AprilTag

The use of AprilTag has several purposes:

- The web camera can be used as a positioning system
- It is useful to dock the robot
- It can be used as a localization if the tag is put on the robot, and a camera is on the ceiling.

6. Object recognition - OpenCV

OpenCV is a library for Computer Vision:

- The web camera can be used for feature detection: person, dog, apple, etc.
- When detecting a person, an alert can be sent to the computer.
- There are several scenarios that can be explored with the use of OpenCV.

7. 3D mapping - OAK-D Lite camera

OAK-D lite is a depth camera based on a stereo camera. The depth information can be used to determine a collision.

- The RTAB Map algorithm, it can stitch and recreate a 3D model of the environment.
- It can help with the odometry (Visual Inertial Odometry, VIO).

8. Visualization for Engineers - Foxglove

RViz has been used to visualize the data; however, this is not the most attractive interface. Foxglove is a WebSocket alternative to be able to communicate with our robot over WebSocket.

9. Visualization for clients - Cloud Computing and Tunneling

RViz and Foxglove are great tools for engineer to debug their system, but quite unappealing.

Designing your own WebSocket with the essential information requires:

- A database to collect and store the data
- Websocket to display this data

To do so, the solution is to host the web server on a cloud computer like a Digital Ocean droplet. However, the WebSocket will be available only locally on the cloud computer. In order to expose it publicly, a tunnel is required, and Cloudflare is the easiest tool available for such a task.

10. Multirobot control - namespace and open-RMF

ROS2 has been designed to control a fleet of robots (as opposed to ROS1). A namespace allows allocating a prefix to the topics, so that there is no conflict when two similar robots are used. The same care must be applied to TF as well. Open-RMF is used to orchestrate the functioning of several robots together. It is especially useful when other types of systems are also in the loop (e.g., elevators, automatic doors, etc.).